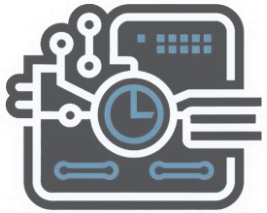


Progettazione Elettronica Digitale

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

L07 – Sommario

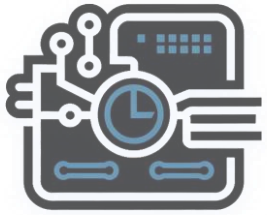
➔Circuiti aritmetici
➔Unità di controllo e data path
➔RTL: Register Transfer Level



Circuiti aritmetici

- I circuiti aritmetici sono un ottimo esempio di progetto di reti logiche
  - Somma binaria mediante half e full adder
  - Circuito moltiplicatore combinatorio
- Decisioni di progetto sull'asse spazio/tempo
  - *Calcolare velocemente* richiede *reti più complesse* e quindi *più spazio*
  - Esempio: circuito di carry look-ahead
- Unità aritmetico-logica
  - Componente fondamentale di processori moderni
  - Usata nel ciclo principale di quasi tutte le istruzioni

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Numerazione binaria

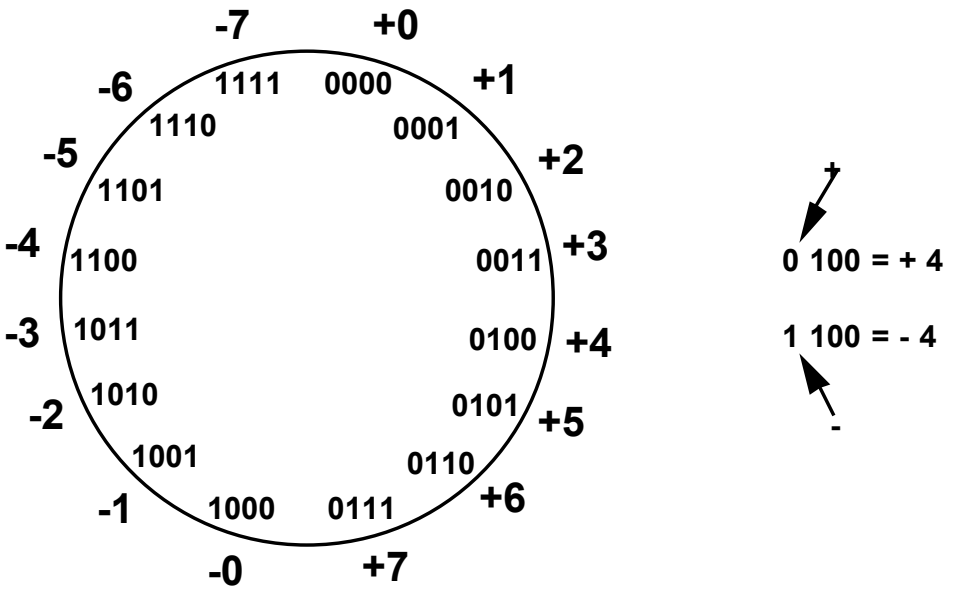
A1

- La rappresentazione dei *numeri positivi* è la stessa in tutti i casi
- La differenza fondamentale sta nella rappresentazione dei *numeri negativi*
  - Modulo e segno
  - Complemento a uno
  - Complemento a due
- Gli esempi a seguire assumono parole di 4 bit che rappresentano 16 valori equamente distribuiti tra positivi e negativi
  - Facilmente estendibile al caso generico di parole a n bit
- Le *diverse rappresentazioni* hanno impatto anche sulla realizzazione delle varie *operazioni aritmetiche*
  → Argomento di approfondimento individuale

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Rappresentazione in modulo e segno

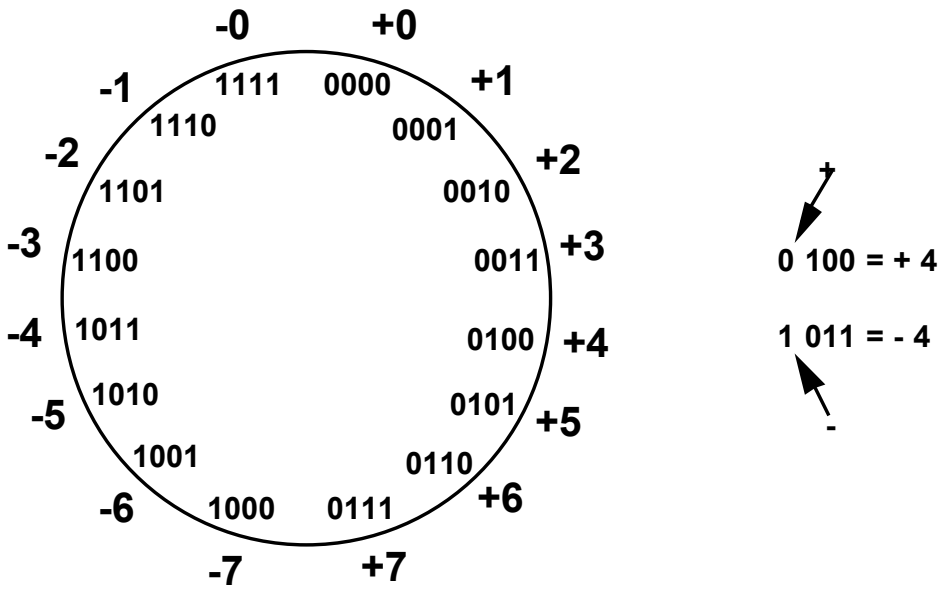
- Bit più significativo (MSB) è il segno
  - $0 =$ positivo (o nullo), $1 =$ negativo
- I restanti bit meno significativi sono il modulo
  - da 0 (000) a 7 (111)
- Intervallo per n bit $= \pm(2^{n-1} - 1)$
- Due rappresentazioni per lo 0
- Somma e sottrazione sono complesse
  - Serve confrontare i moduli per *decidere il segno*



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Rappresentazione in complemento a 1 (CA1)

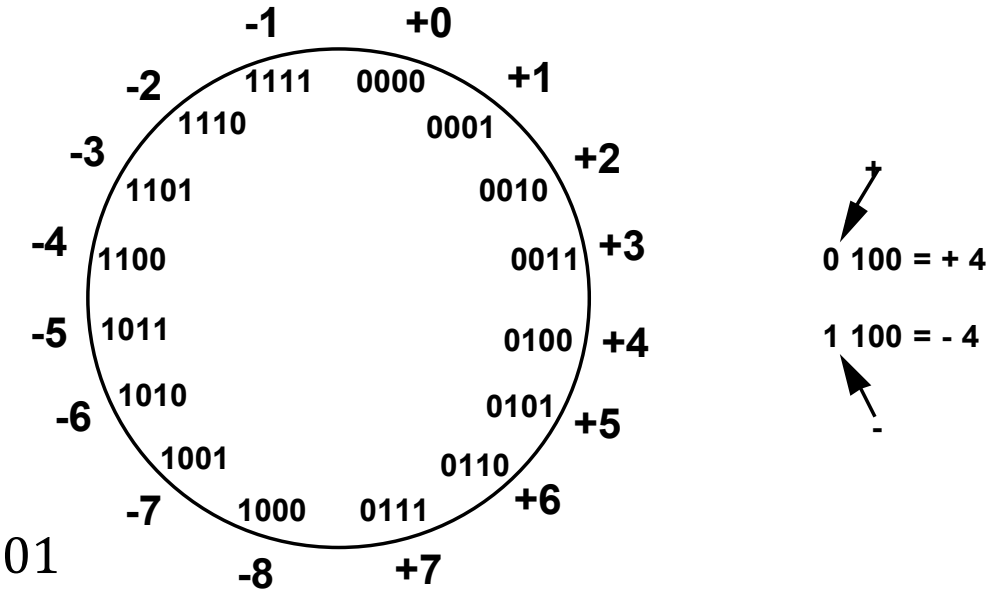
- Se N è un numero positivo, $\overline{N}$ è l'opposto ottenuto tramite CA1
- $\overline{N} = (2^n - 1) - N$
  - Equivale a calcolare il *complemento bit a bit*
- Ancora due rappresentazioni per lo 0
- Sottrazione realizzata come CA1 e somma
  - Complicazioni per la somma



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Rappresentazione in complemento a 2 (CA2)

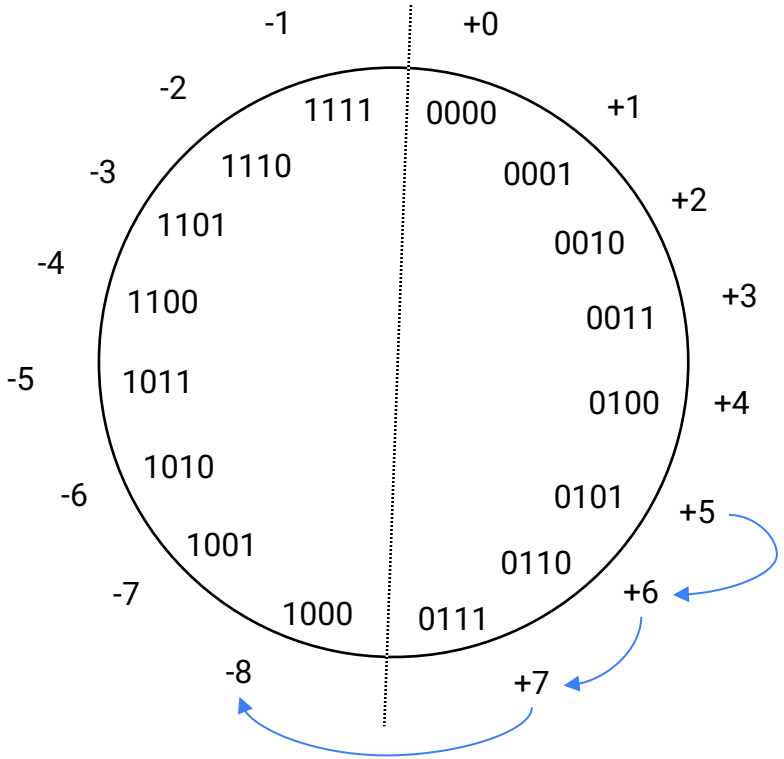
- Come CA1, con una rotazione in senso orario di uno passo per i negativi
  - CA1: $(-7, 1000)$, $(-6, 1001)$, $(-5, 1010)$, …
- Solo *una rappresentazione per lo* 0
  - Un numero negativo in più rappresentabile
- $\overline{N} = (2^n - N)$
  - Metodo abbreviato di calcolo per $\overline{N}$
    - CA1 tramite complemento bit-a-bit
    - Somma di 1 al valore in CA1
- Esempi:
  - $+7 \rightarrow -7$: $0111 \rightarrow \textbf{CA1}(0111) + 1 = 1000 + 1 = 1001$
  - $-7 \rightarrow +7$: $1001 \rightarrow \textbf{CA1}(1001) + 1 = 0110 + 1 = 0111$



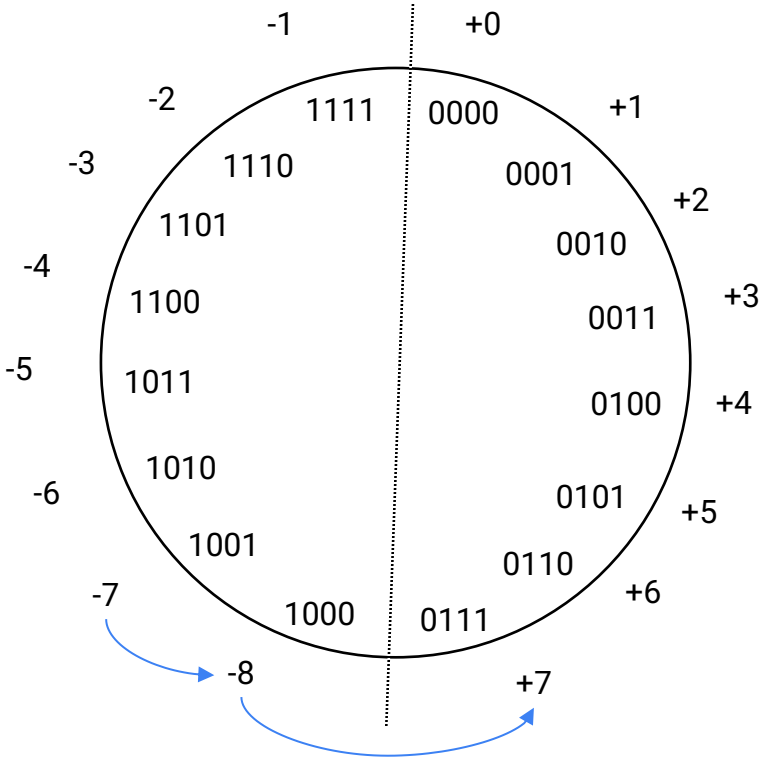
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Condizioni di overflow in CA2

- Overflow: sommare due numeri *negativi* e ottenerne uno *positivo*, o viceversa



$5 + 3 \rightarrow -8$



$-7 - 2 \rightarrow +7$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Condizioni di overflow in CA2

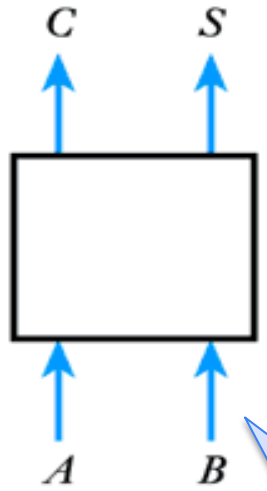
- Si ha *overflow* quando il *riporto sul segno* e il *riporto in uscita* differiscono tra di loro

	0 1 1 1				**1 0** 0 0		
5	0 1 0 1	-7	1 0 0 1	5	0 1 0 1	-3	1 1 0 1
3	0 0 1 1	-2	1 1 0 0	2	0 0 1 0	-5	1 0 1 1
-8	1 0 0 0	7	1 0 1 1 1	7	0 1 1 1	-8	1 1 0 0 0
Overflow		Overflow		Non è overflow		Non è overflow	

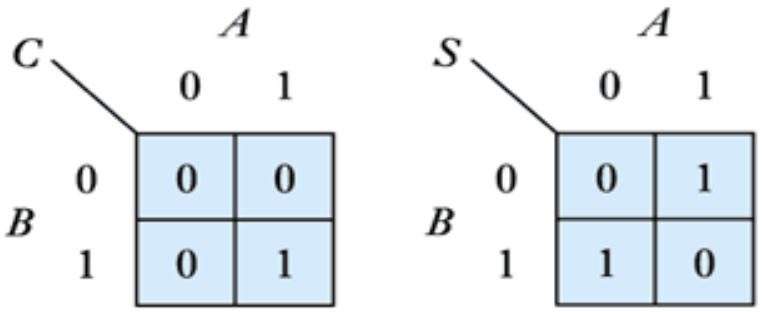
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Reti per aritmetica binaria

- Molto più semplice che l'aritmetica decimale
  - Sono necessari solo circuiti relativamente semplici
- Se consideriamo il CA2, è sufficiente un half adder (semisommatore)



A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

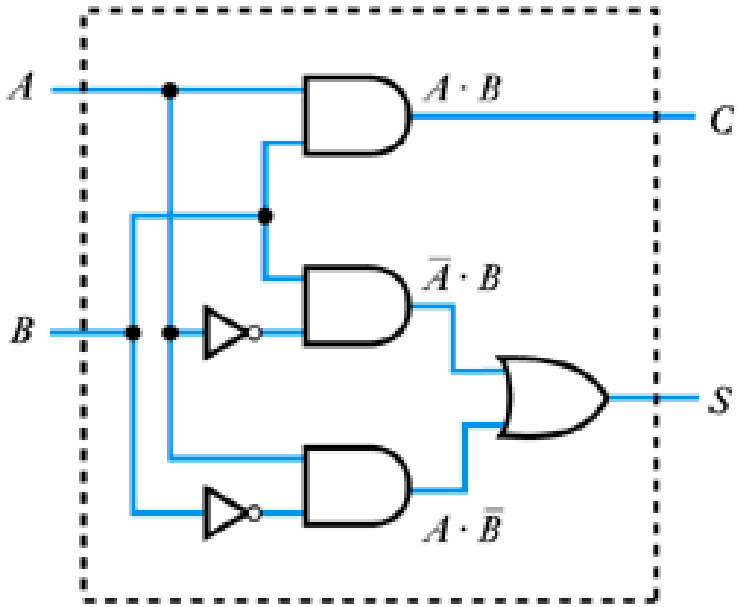


Qui servirà anche un riporto in ingresso

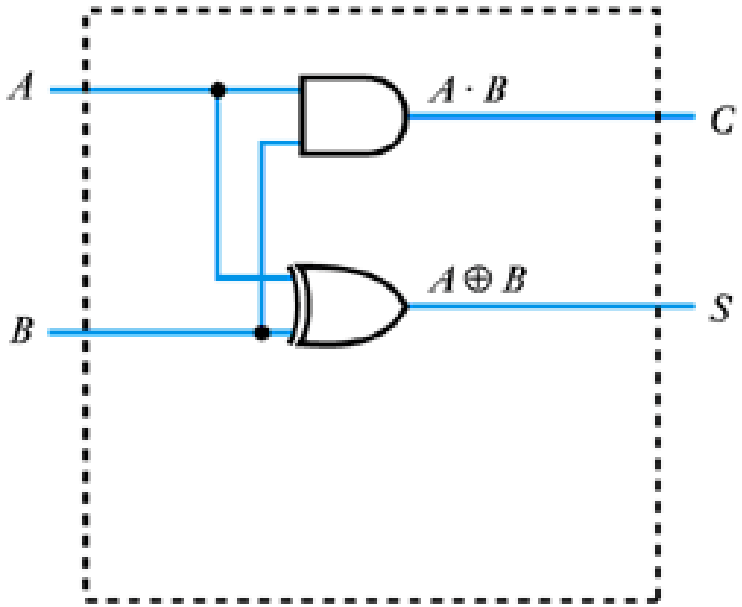
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Implementazione di half adder

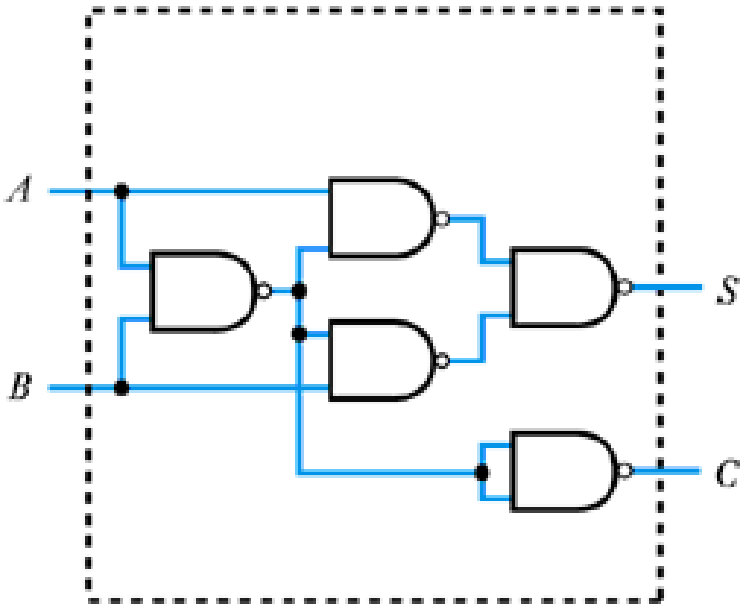
- Il semisommatore ha varie implementazioni equivalenti a livello funzionale



(a)



(b)



(c)

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Dall’half al full adder e alle somme multibit

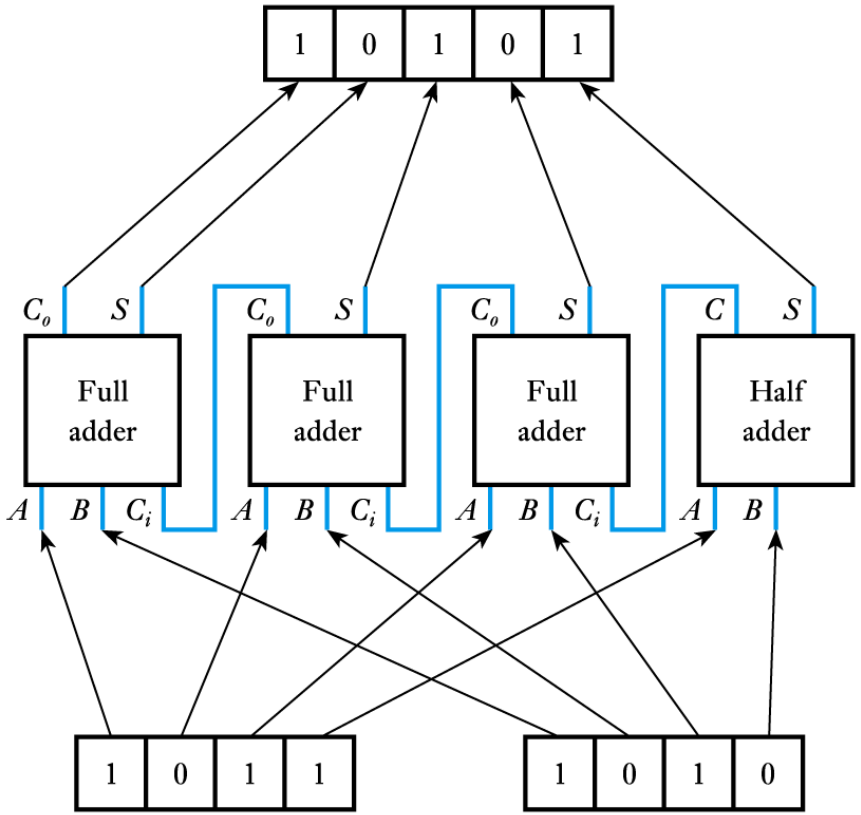
- Un sommatore generico deve solitamente gestire anche un riporto in ingresso (carry-in)

➔ *Full adder*

  - Combinazione di due half adder

➔ *Somme multibit* con una *catena di full adder*

  - Un full adder per ogni *posizione* della parola binaria
  - Il *riporto transita in cascata* dal least (LSB) al most significant bit (MSB)

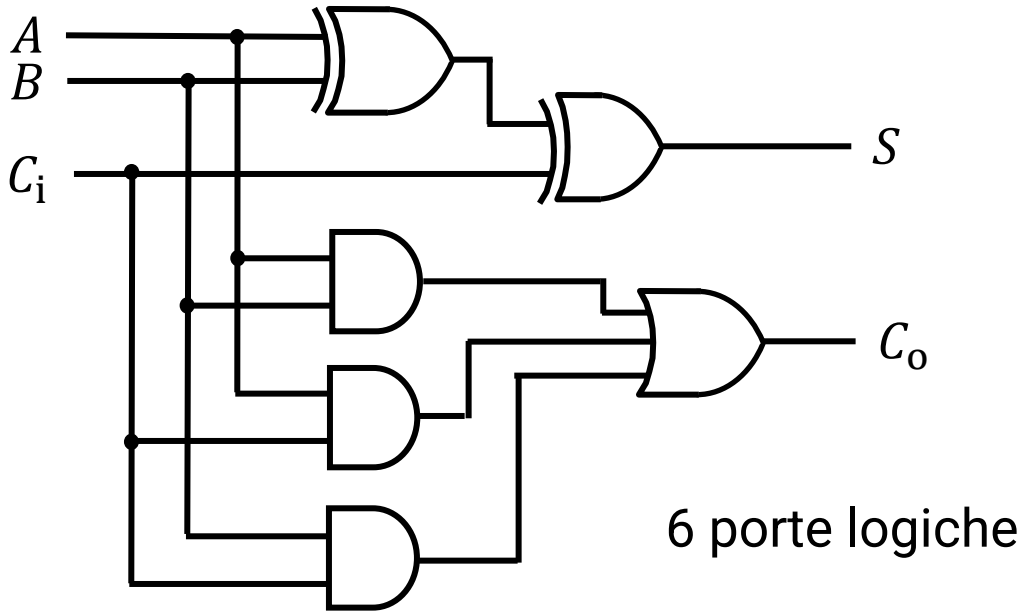


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Full adder digitale

- L'*unità base* di un sommatore multi-bit è il *full adder*
- Input
  - *Addendi* e *riporto* in ingresso
- Output
  - *Somma* e *riporto* in uscita

A	B	C_i	S	C_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



$$S = A \oplus B \oplus C_\mathrm{i}$$

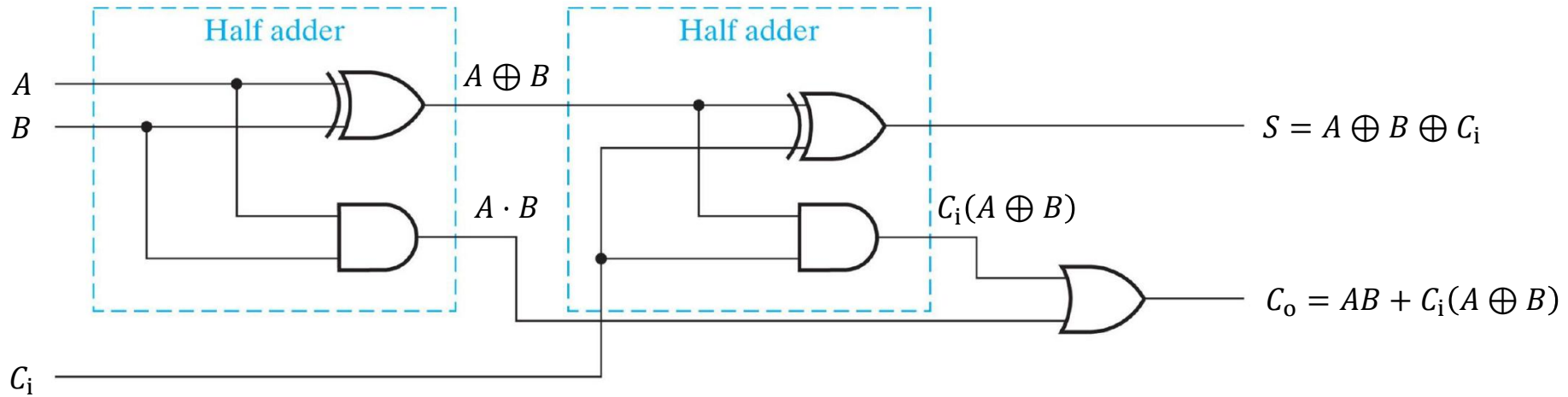
$$C_\mathrm{o} = A \cdot B + A \cdot C_\mathrm{i} + B \cdot C_\mathrm{i} = A \cdot B + C_\mathrm{i}(A + B)$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Full adder realizzato con half adder

- Si può implementare un full adder con solo 5 porte,
  con *due half adder in catena* e *una porta OR*
- Osservazione: in $C_\mathrm{o} = A \cdot B + C_\mathrm{i}(A + B)$, $(A + B)$ può essere sostituito con $(A \oplus B)$
  $C_\mathrm{o} = A \cdot B + C_\mathrm{i}(A \oplus B)$
  - $A + B \neq A \oplus B$ solo quando $A = B = 1$
  - Ma quando $A = B = 1 \Longrightarrow C_\mathrm{o} = 1$ già perché il primo termine $A \cdot B = 1 \cdot 1 = 1$,
    a prescindere dal valore potenzialmente errato (se $C_\mathrm{i} = 1$) del termine $C_\mathrm{i}(A \oplus B)|_{A=B=1}$

$$C_\mathrm{o}\Big|_{A=B=1} = 1 \cdot 1 + C_\mathrm{i}(1 \oplus 1) = 1$$

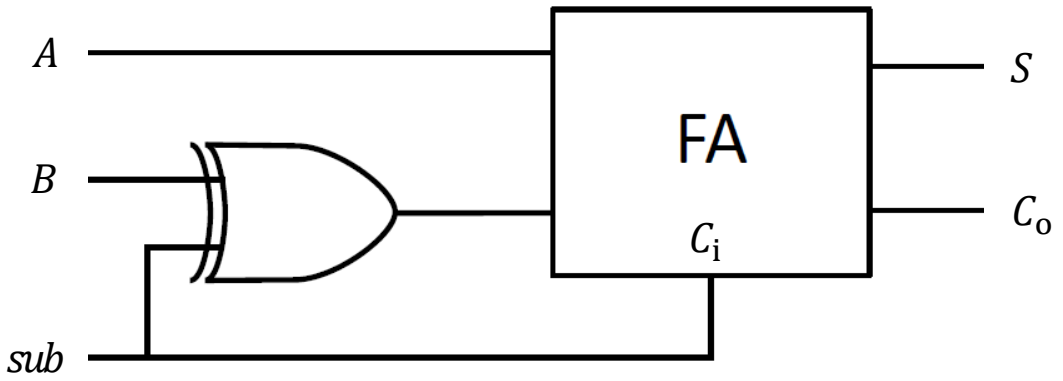


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Addizionatore/Sottrattore binario

A2

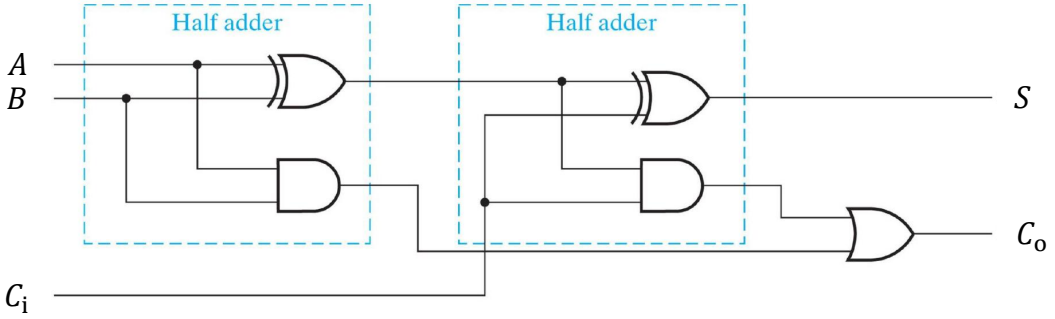
- Con le regole della CA2, un full adder può calcolare sia la *somma* sia la *differenza*
- La porta XOR nello schema sotto
  - *Seleziona* l'operazione: somma o differenza
  - Gestisce il *valore del secondo addendo*
  - Se $sub = 0$ esegue la *somma*
    - La porta XOR trasmette B e il full adder calcola $A + B$ *senza riporto in ingresso* perché $C_{\mathrm{i}} = sub = 0$
  - Se $sub = 1$ esegue la *differenza*
    - La porta XOR trasmette il *complemento di* B
      → il full adder calcola $A + \boldsymbol{CA2}(B) = A + \left(\overline{B} + 1\right) = A - B$

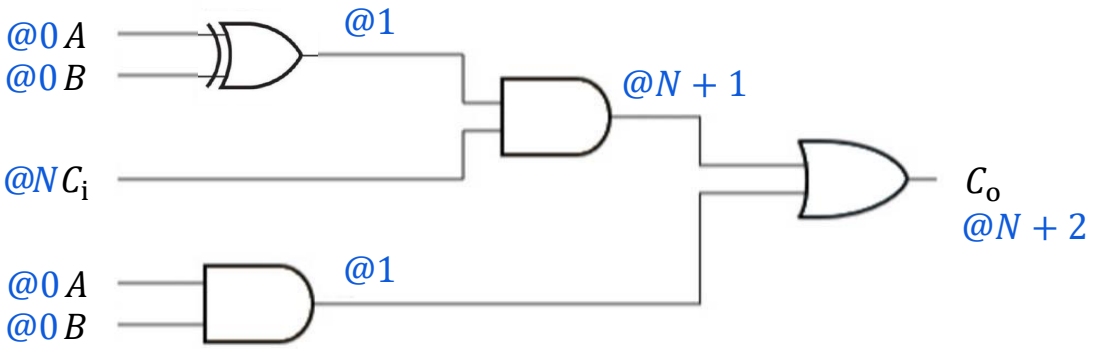


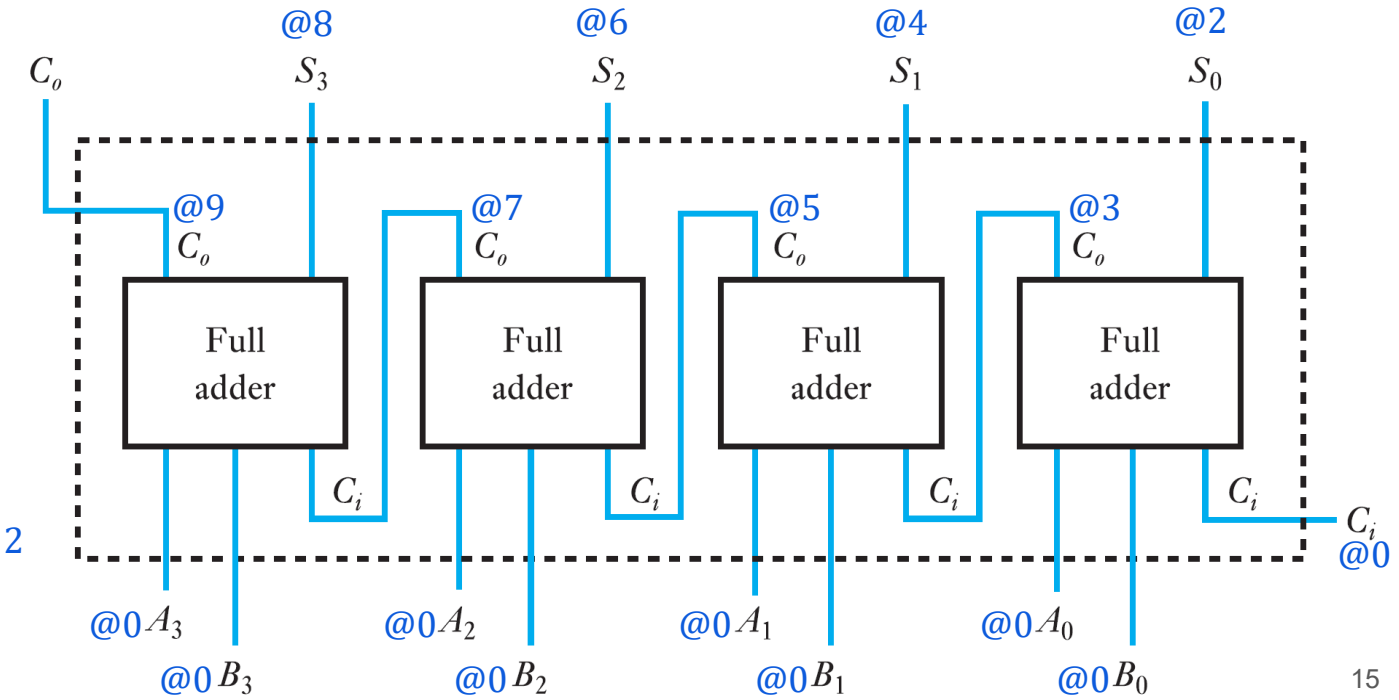
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Temporizzazione per adder ripple carry

- Nelle somme multibit con una catena di full adder, la *propagazione del riporto* dagli stati precedenti (ripple carry) introduce un *ritardo critico*
  - Indichiamo con $@X$ il tempo di «arrivo» del dato su un segnale in un punto del circuito
  - Assumiamo che ogni elemento introduca un *ritardo unitario*



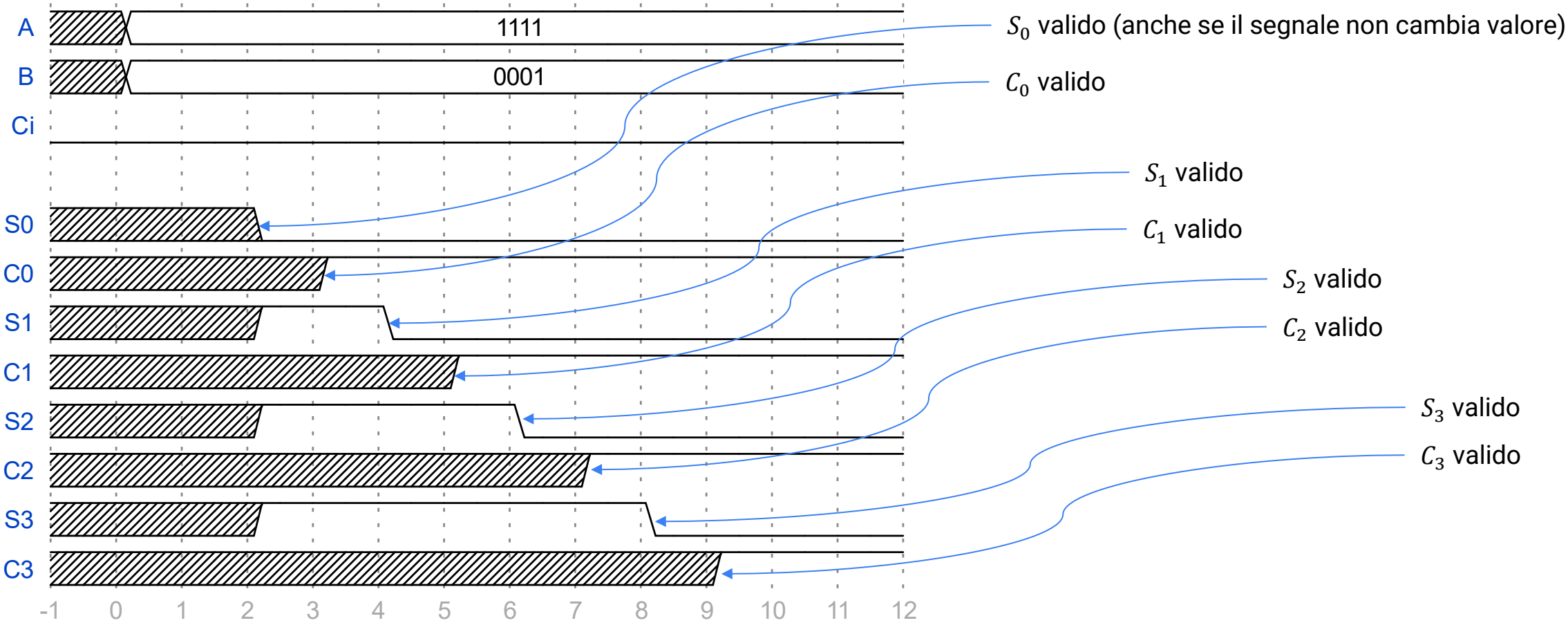




2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Temporizzazione per adder ripple carry

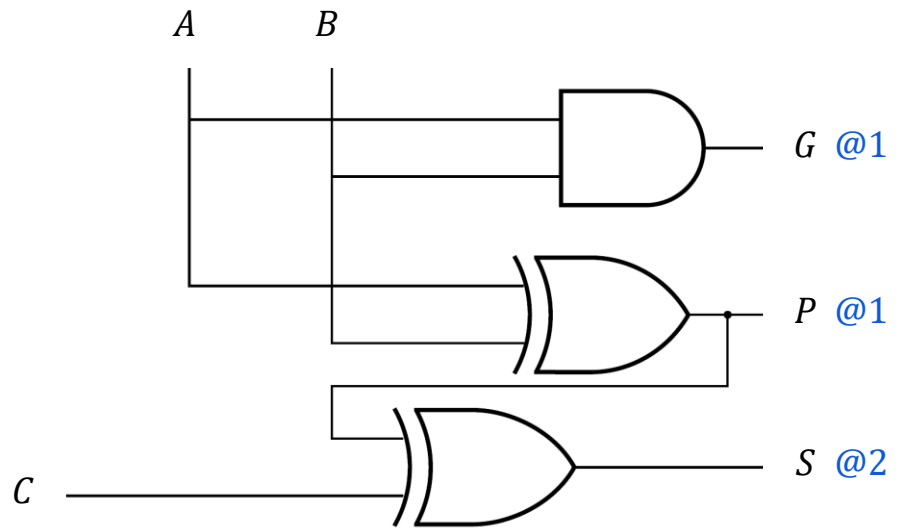
- Caso peggiore (massimo ritardo): $1111 + 0001$
  - Tutti gli output intermedi devono assestarsi prima che sia valido l'MSB
  - Ritardo dettato dal percorso «più lungo»



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

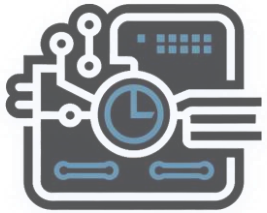
Circuito di carry lookahead

- Per ridurre il ritardo, si introduce una struttura di *carry lookahead* parallela, più complessa
- Si introducono due espressioni di comodo
  - *Carry generate* definita come $G_i = A_i \cdot B_i$
  - *Carry propagate* definita come $P_i = A_i \oplus B_i$
- Nella cella di *carry lookahead*
  - Non c'è riporto in uscita
  - P e G *non dipendono* dal carry in ingresso
- La *somma* e il *riporto* possono essere espresse in funzione di *carry generate* e *propagate*



$$S_i = A_i \oplus B_i \oplus C_i = P_i \oplus C_i$$
$$C_{i+1} = A_iB_i + A_iC_i + B_iC_i = \cdots = A_iB_i + C_i(A_i \oplus B_i) = G_i + C_iP_i$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Circuito di carry lookahead

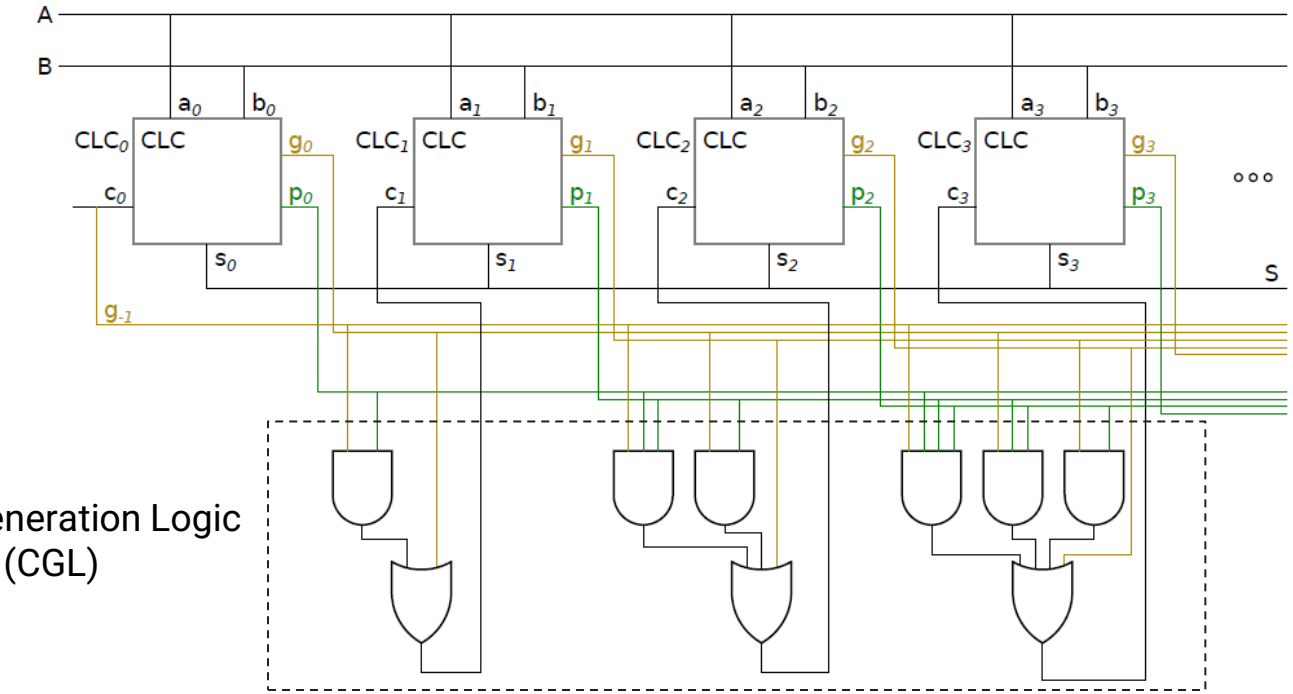
- Si possono riscrivere le espressioni della logica del riporto in funzione carry generate ($G_i = A_i \cdot B_i$) e carry propagate ($P_i = A_i \oplus B_i$)
  - $C_1 = G_0 + P_0 C_0$
  - $C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_0$
  - $C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$
  - $C_4 = G_3 + P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$
  - …
  - $C_i = \sum_{j=-1}^{i-1} G_j \prod_{k=j+1}^{i-1} P_k$ (con $C_0 = G_{-1}$)
- Ogni equazione può essere espressa mediante una *rappresentazione su due livelli*
- *Variabili di ingresso*
  - Addendi A_i, B_i
  - Riporto in ingresso allo *stadio zero* C_0

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Circuito di carry lookahead

- La logica di lookahead si fa man mano più complicata
- Il cammino più lungo è costante (2 livelli) se realizzato con *porte CMOS complesse*
  - Ma le porte CMOS complesse, con N ingressi, diventano lente per $N > 4$…

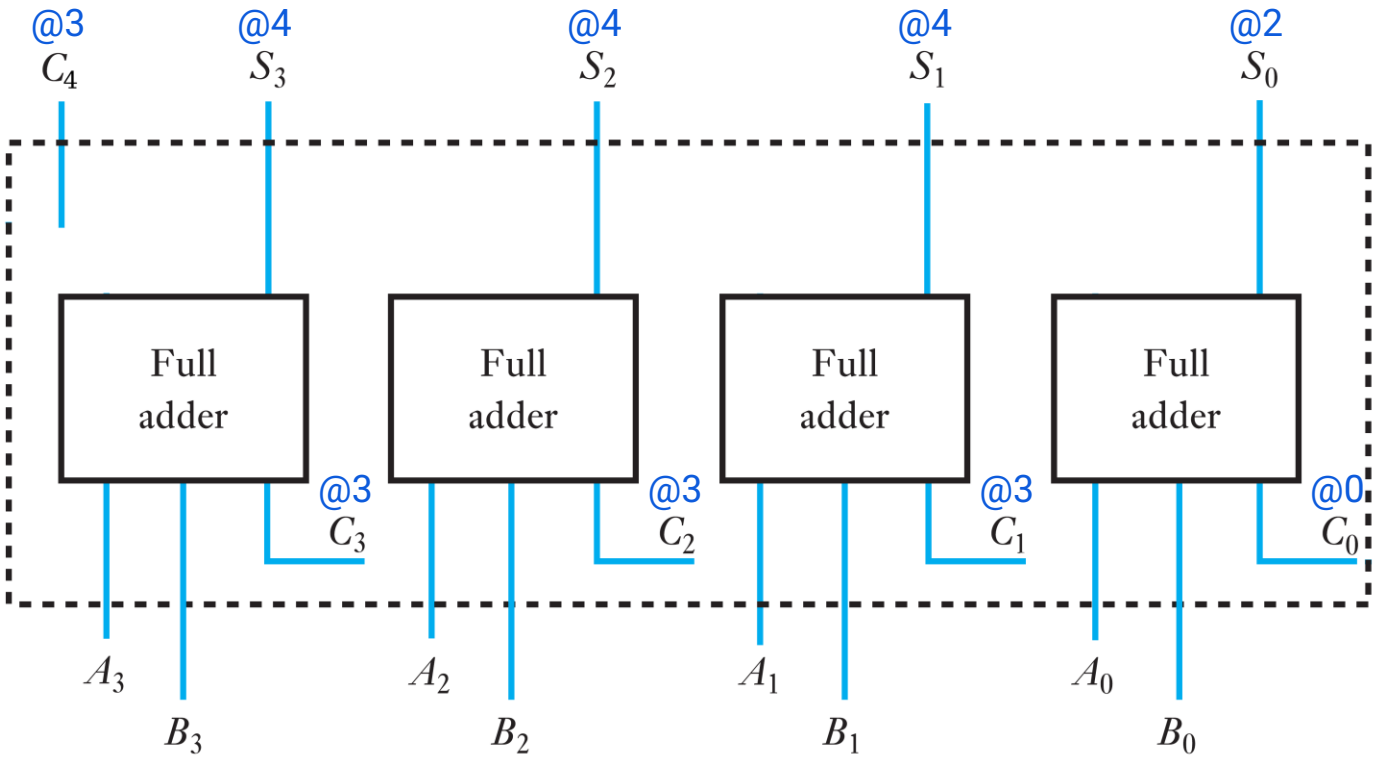
➔ CLC: carry lookahead cell, dedicata al calcolo di p e g, e della somma s



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Circuito di carry lookahead

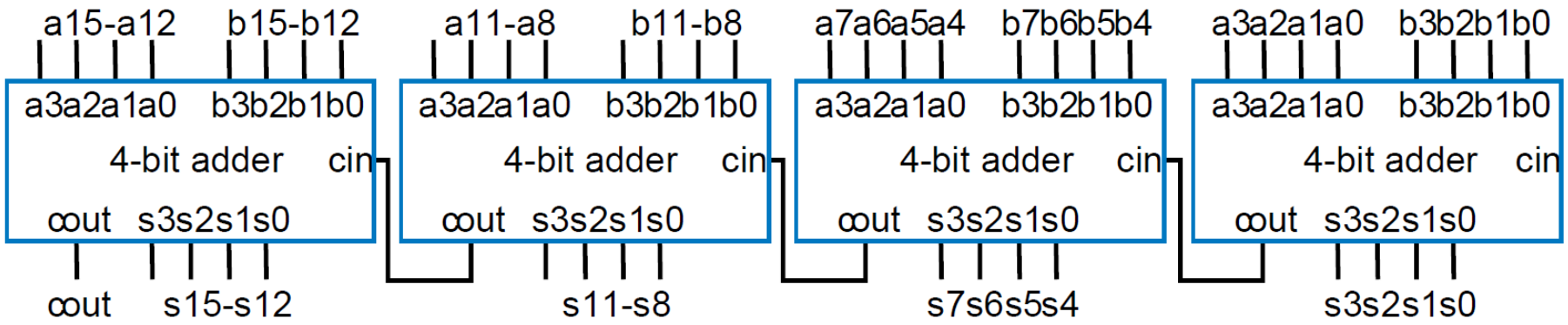
- La logica di lookahead genera tutti i riporti allo *stesso momento*
- Le somme possono essere calcolate in parallelo (più rapidamente)
  - Il calcolo del carry subisce il ritardo della cella CLC (calcolo di p_i e g_i) più quello della rete CGL



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Ripple CLA

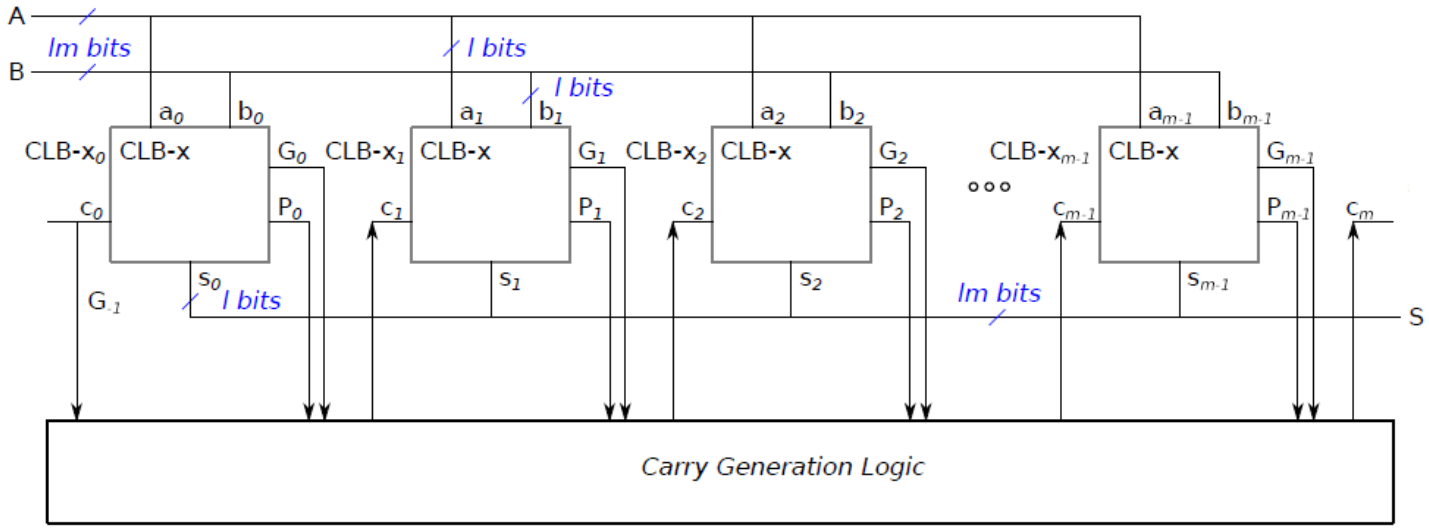
- Problema CLA: al crescere del numero di stadi, cresce il numero di gate e il rispettivo fan-in
  - 5-in gate al quarto stadio
  - 33-in gate al trentaduesimo stadio
    - Fan-in troppo elevato per essere utilizzato direttamente
    - Sostituito da una struttura a *più livelli*, sfruttando *porte più piccole*
- Una soluzione: collegare più CLA in cascata, come per il ripple carry
  - Ma così si ha di nuovo un ritardo dovuto al *collegamento in cascata* dei moduli



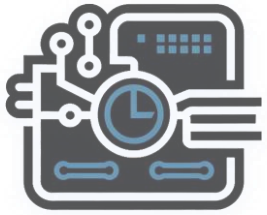
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

CLA gerarchico

- Un CLA gerarchico applica su *più livelli* la logica di carry lookahead, utilizzando multipli blocchi «elementari» con un dato parallelismo
- I *carry lookahead block (CLB)* possono includere qualsiasi tipo di sommatore
  - CLB-r (ripple)
  - CLA-c (carry lookahead)
- Il *carry generation logic* (CGL o CLL: carry lookahead logic) può a sua volta essere ripple o lookahead



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



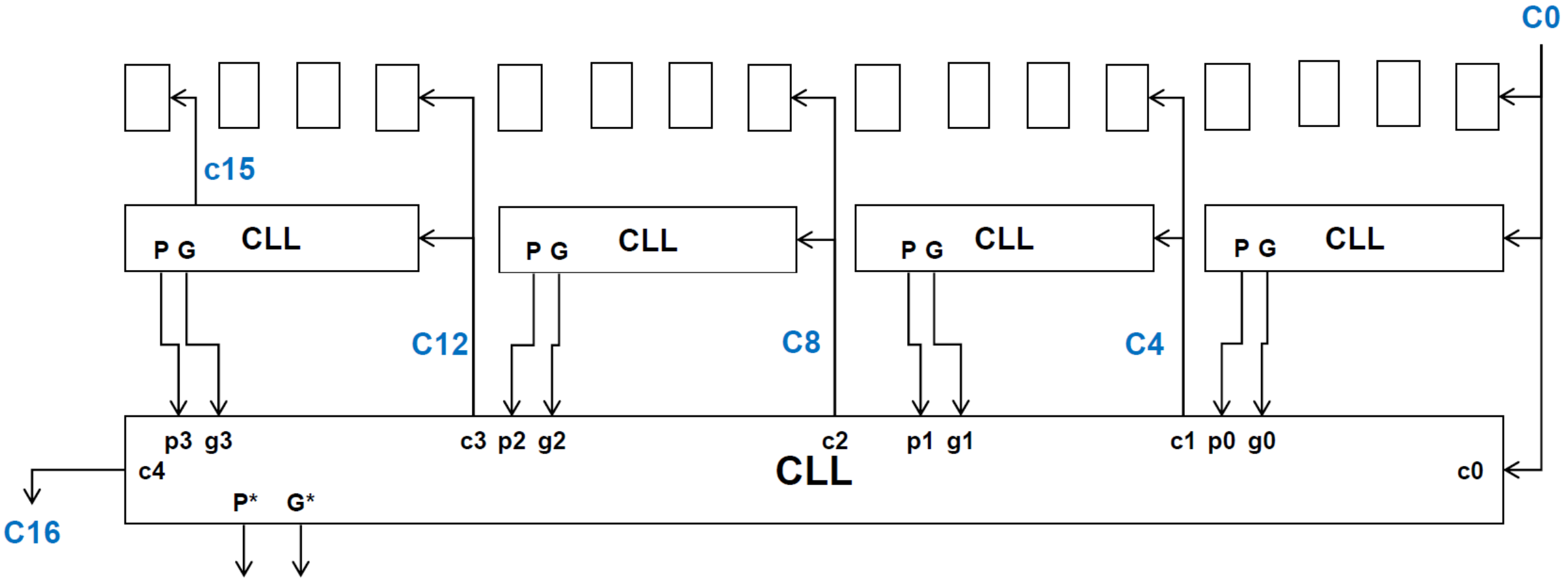
CLA gerarchico
Esempio a 16 bit

- In un adder a 16 bit, che usa 4 CLA a 4 bit ciascuno, valgono le considerazioni
  - Ogni CLA a 4 bit *propaga il proprio carry* se tutte le sue colonne propagano
    - $P0 = p_3 \cdot p_2 \cdot p_1 \cdot p_0$
    - $P1 = p_7 \cdot p_6 \cdot p_5 \cdot p_4$
    - …
  - Ogni CLA a 4 bit *genera un carry* se una qualsiasi colonna genera un carry e si ha propagazione sulla cifra più significativa
    - $G0 = g_3 + p_3 \cdot g_2 + p_3 \cdot p_2 \cdot g_1 + p_3 \cdot p_2 \cdot p_1 \cdot g_0$
    - $G1 = g_7 + p_7 \cdot g_6 + p_7 \cdot p_6 \cdot g_5 + p_7 \cdot p_6 \cdot p_5 \cdot g_4$
    - …
  - La logica di ordine superiore per la generazione dei carry ($C4$, $C8$, $C12$, $C16$) può seguire le stesse regole viste in precedenza per il circuito di carry lookahead
    - $C4 = G0 + P0 \cdot c_0$
    - $C8 = G1 + P1 \cdot G0 + P1 \cdot P0 \cdot c_0$
    - …

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

CLA gerarchico
Esempio a 16 bit

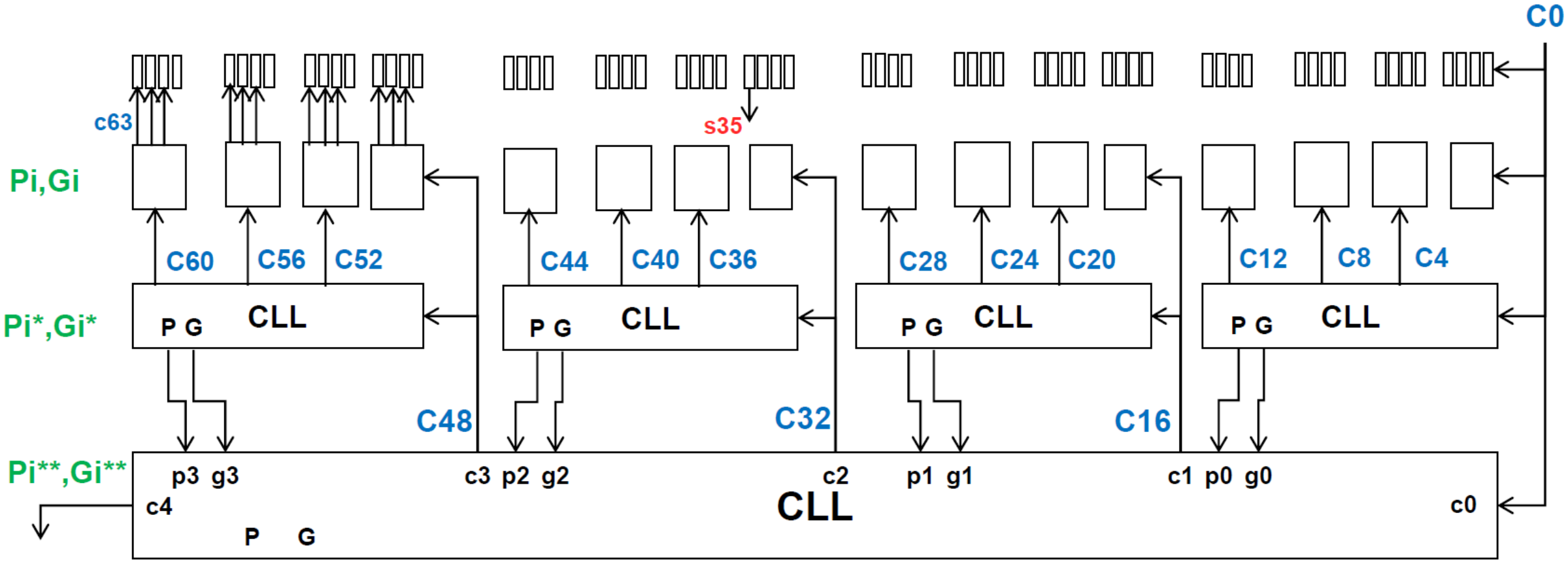
- Si usa un'organizzazione gerarchica di CLL che via via estende la logica vista in precedenza a porzioni di ordine superiore del circuito



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

CLA gerarchico
Esempio a 64 bit

- Si usa un'organizzazione gerarchica di CLL che via via estende la logica vista in precedenza a porzioni di ordine superiore del circuito



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

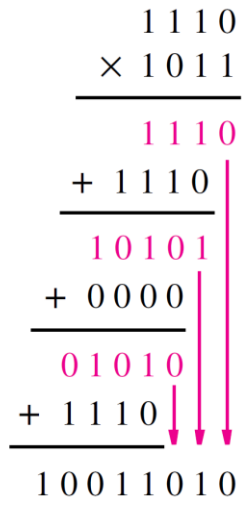
Moltiplicazione binaria

- La moltiplicazione binaria, per numero senza segno, si può impostare esattamente come quella in base decimale
  - Si esaminano i bit da sinistra a destra
  - Se un bit è a 1, si aggiunge una versione opportunamente shiftata del moltiplicando a un prodotto parziale
  - Se un bit è a 0, non dà contributo
  - Il prodotto finale è ottenuto sommando in colonna tutti i prodotti parziali

$M\ (14_{10})$ 1 1 1 0
$Q\ (11_{10})$ × 1 0 1 1

1 1 1 0
1 1 1 0
0 0 0 0
1 1 1 0

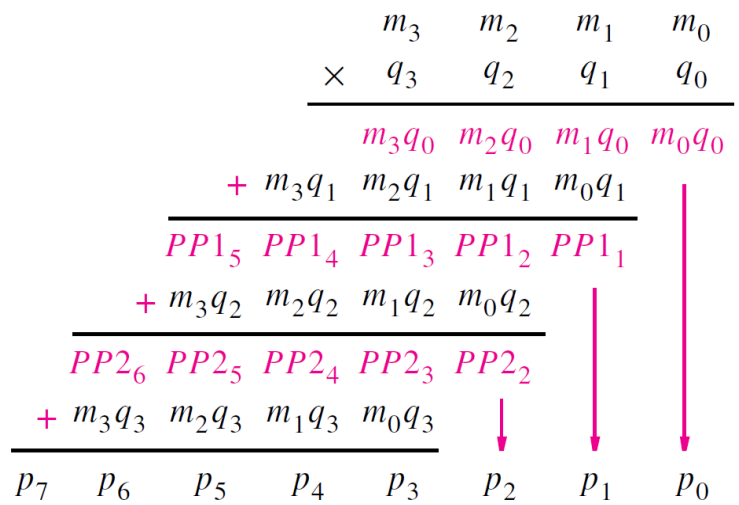
$P\ (154_{10})$ 1 0 0 1 1 0 1 0



prodotto parziale 0

prodotto parziale 1

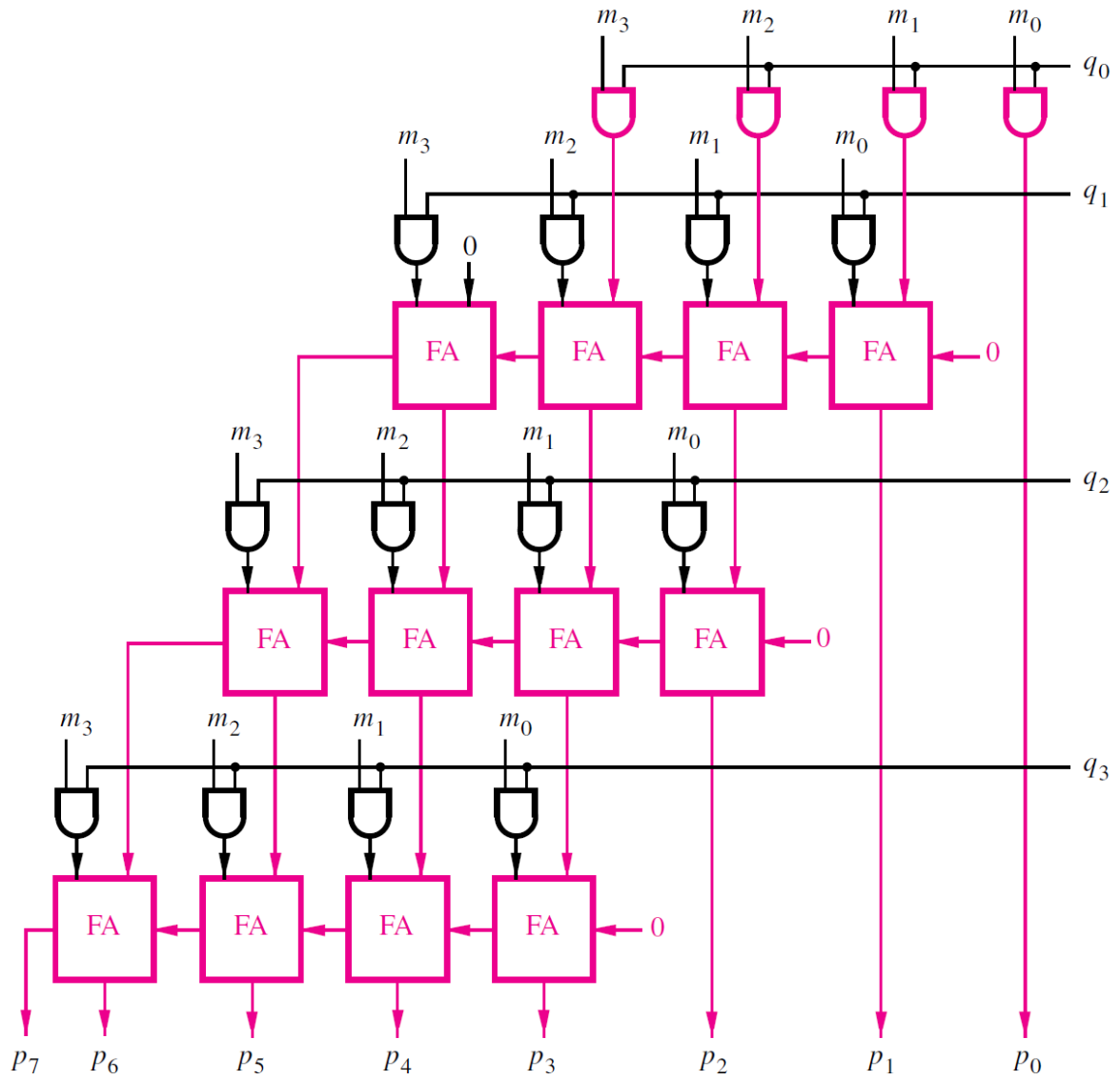
prodotto parziale 2



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Moltiplicatore parallelo

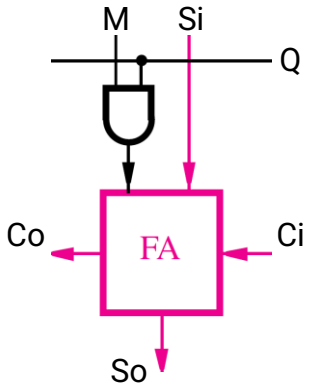
$$
\begin{array}{rcccc}
 & m_3 & m_2 & m_1 & m_0 \\
\times & q_3 & q_2 & q_1 & q_0 \\
\hline
 & m_3q_0 & m_2q_0 & m_1q_0 & m_0q_0 \\
+ & m_3q_1 & m_2q_1 & m_1q_1 & m_0q_1 \\
\hline
 & PP1_5 & PP1_4 & PP1_3 & PP1_2 \quad PP1_1 \\
+ & m_3q_2 & m_2q_2 & m_1q_2 & m_0q_2 \\
\hline
 & PP2_6 & PP2_5 & PP2_4 & PP2_3 \quad PP2_2 \\
+ & m_3q_3 & m_2q_3 & m_1q_3 & m_0q_3 \\
\hline
p_7 & p_6 & p_5 & p_4 & p_3 \quad p_2 \quad p_1 \quad p_0
\end{array}
$$

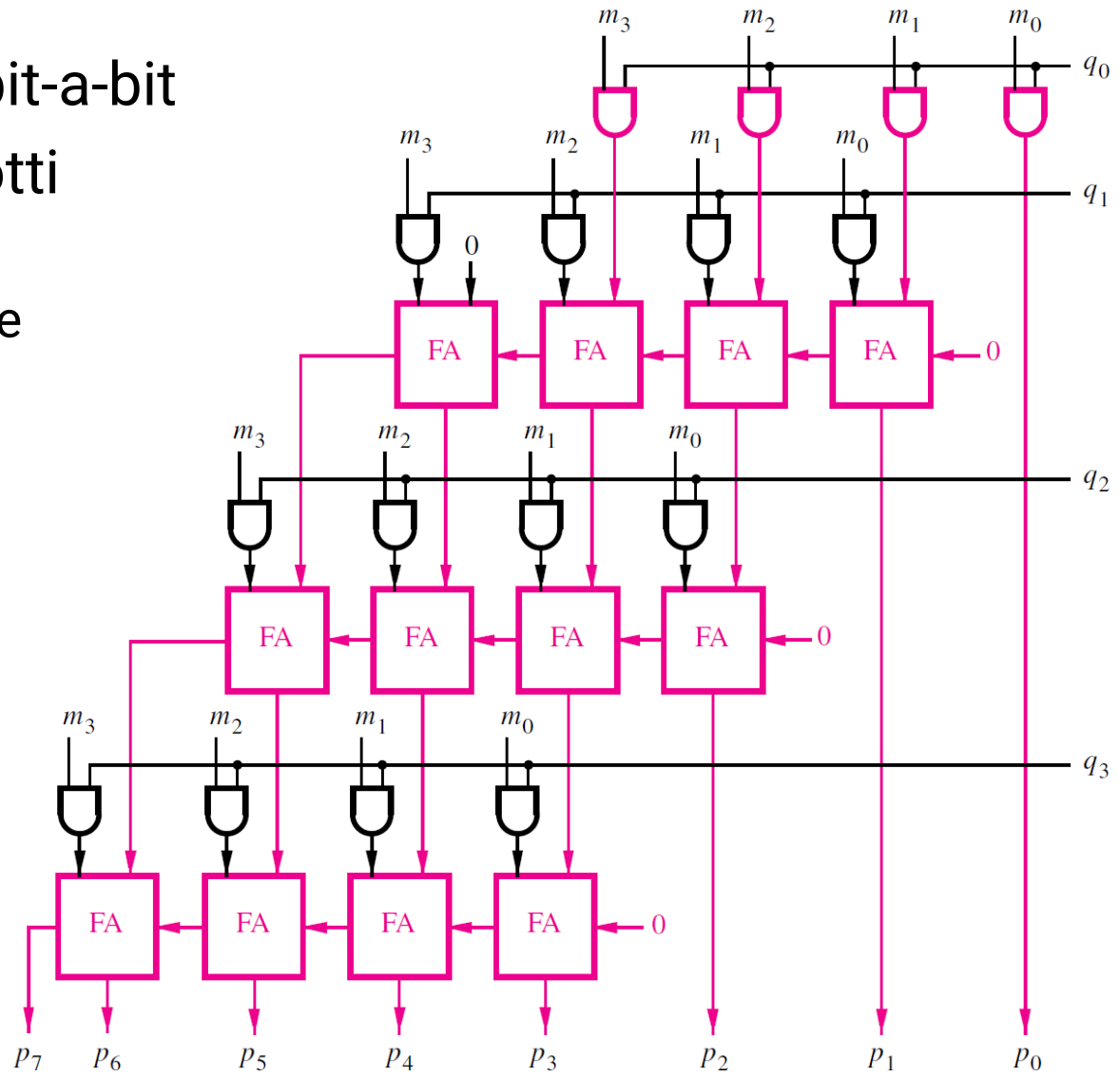


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

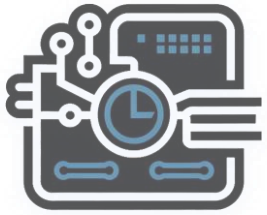
Moltiplicatore parallelo

- 16 porte AND per calcolare i prodotti bit-a-bit
- 12 sommatori per la somma dei prodotti parziali e del risultato finale
  - Se sono FA, usano indicativamente 6 porte
- Per le somme di ordine più elevato si sfruttano più carry-out in parallelo
- Si può identificare un «blocco base» composto da un FA e una porta AND





2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



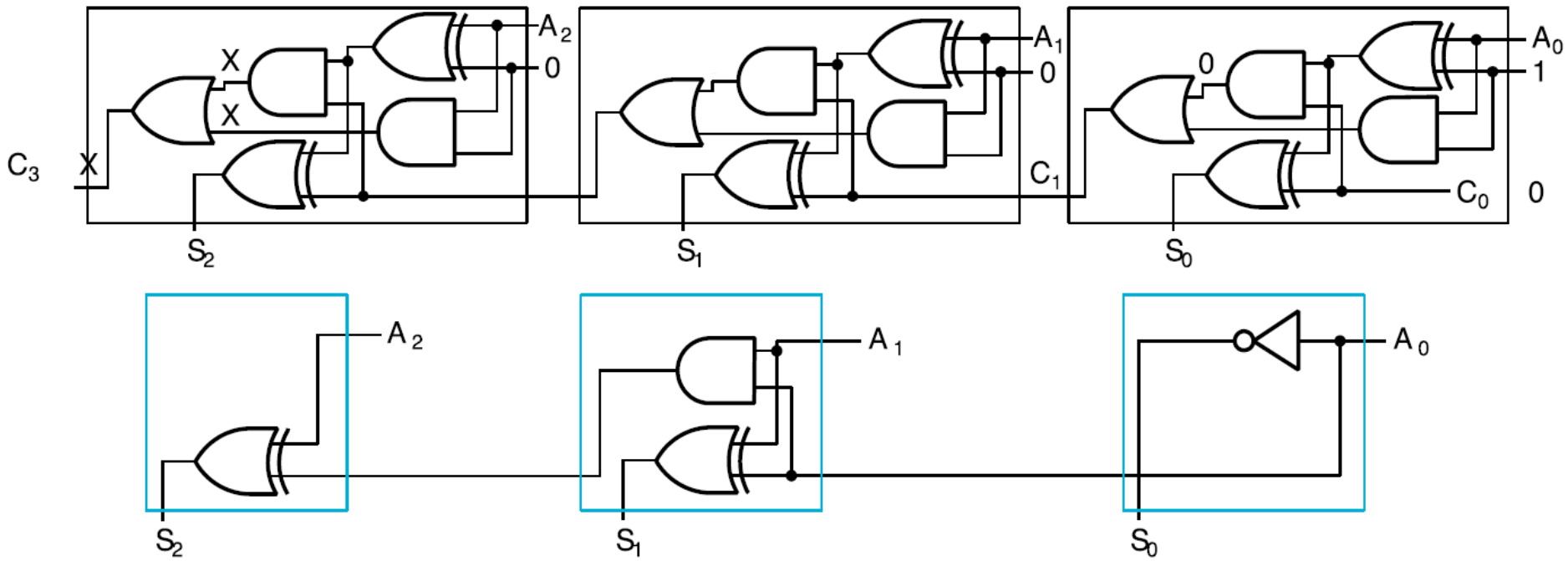
Altre funzioni aritmetiche

- Risulta spesso conveniente definire l'implementazione di nuovi blocchi funzionali per «contrazione», ossia la rimozione di circuiteria ridondante laddove alcuni input siano stati resi delle costanti
  - Incremento/decremento (costante ± 1)
  - Moltiplicazione/divisione per una costante
  - Estensione del segno e zero-fill
- La nuova funzione deve essere ottenibile da quella originale, eventualmente con della (semplice) logica aggiunta ai suoi ingressi
- Si tratta di una forma di pruning
  - Alcuni input vengono fissati $(0/1)$
  - Alcune uscite diventano irrilevanti (don't care)
  - A seguire, la rappresentazione corrispondente può essere semplificata

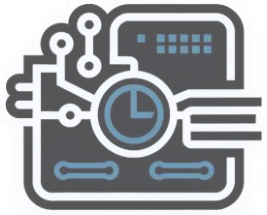
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio: incremento su 3 bit

- Si trasforma un sommatore ripple carry a 3 bit in un componente per calcolare incremento unitario
  - A rimane tale e quale
  - B è fisso a 001
- Si può estendere a un parallelismo arbitrario replicando la cella intermedia



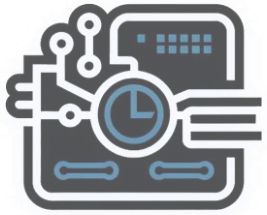
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Circuiti aritmetici e sistemi complessi

- Fino adesso si sono considerati circuiti aritmetici dedicati alla realizzazione di una funzione specifica
- Nei sistemi digitali possono essere
  - Tante unità che implementano funzioni specifiche
  - Interconnesse tra loro e comandate da segnali per
    - Selezione dell'operazione da svolgere
    - I valori in ingresso sui quali svolgerla
- Vari criteri per selezionare e svolgere le operazioni
  - Stato del sistema
  - Temporizzazione
  - Altre condizioni
- L'obiettivo è distinguere
  - La parte di «*calcolo*»
  - La logica dedicata al *controllo* del sistema stesso

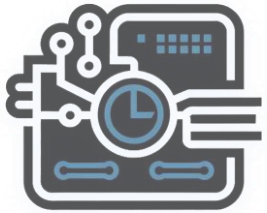
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



FSM e sistemi complessi

- Le FSM possono modellare varie tipologie di algoritmo e sistemi
- Una FSM può rappresentare un *qualsiasi sistema discreto* che *evolve nel tempo*, attraversando un *numero finito di stati* e con un *numero finito di ingressi e uscite*
- È *estremamente generale e versatile*, ma alcuni sistemi possono essere *troppo complessi* per essere *modellati efficacemente* in questo modo
  - Un *registro a scorrimento* a 8 bit è descrivibile con una FSM che comprende 256 *stati*
  - Un registro a 32 bit ha uno spazio degli stati con 2^{32} *stati differenti*
- Separando il ruolo di *controllore* (logica) da altre mansioni, come *calcoli aritmetici*,
  → Si può *ottimizzare il progetto e la realizzazione* di alcuni sistemi digitali

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



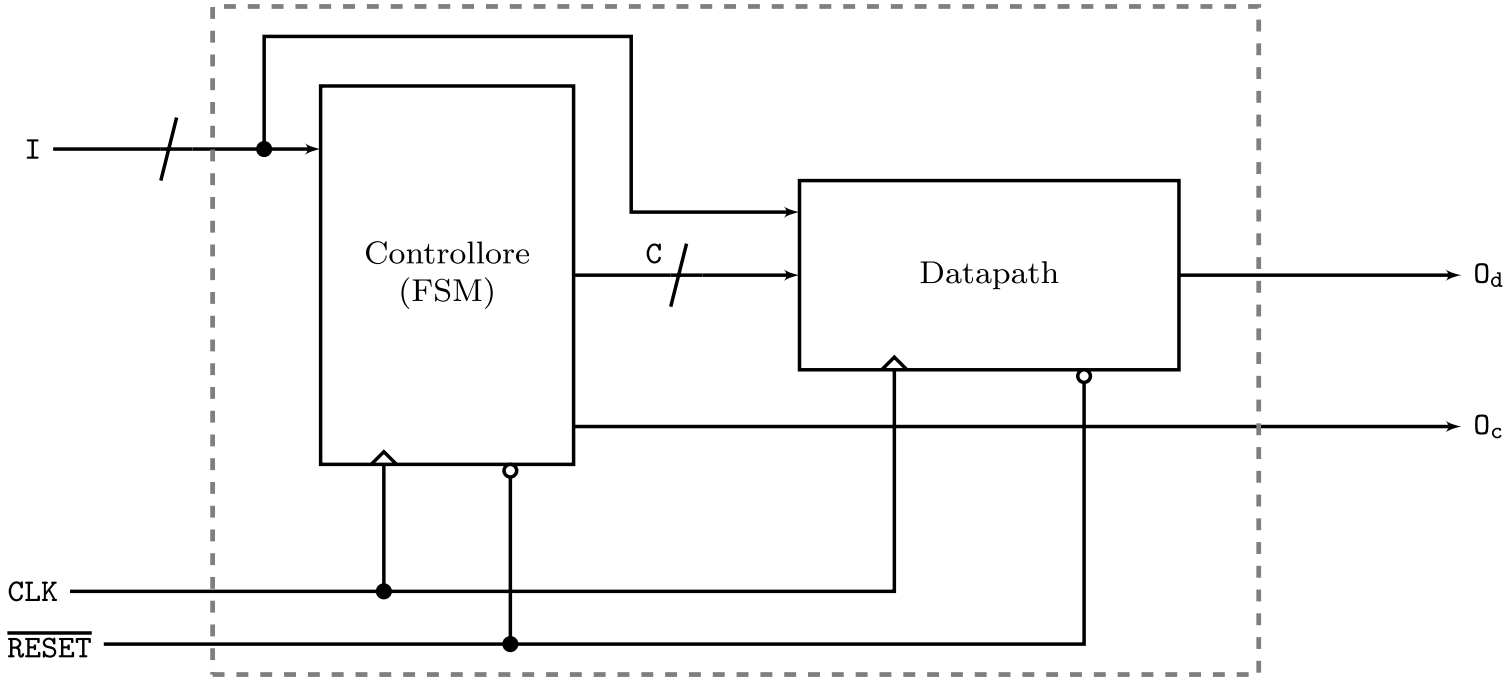
FSM come controllore

- Il *controllore* (*controller*), progettato come *FSM*, ha il compito di *gestire il funzionamento complessivo* del sistema
  - Nomi meno frequenti
    - *Sequenziatore* (*sequencer*)
    - *Temporizzatore* (*timing generator*)
- I componenti dedicati definiscono l'*architettura (architecture) del sistema*, che *interagisce direttamente con i dati*
- L'architettura viene indicata in generale con il termine *«percorso dei dati»* (*datapath*)
- La coppia più classica di termini è *controllore e datapath*

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Sistemi a controllo aperto

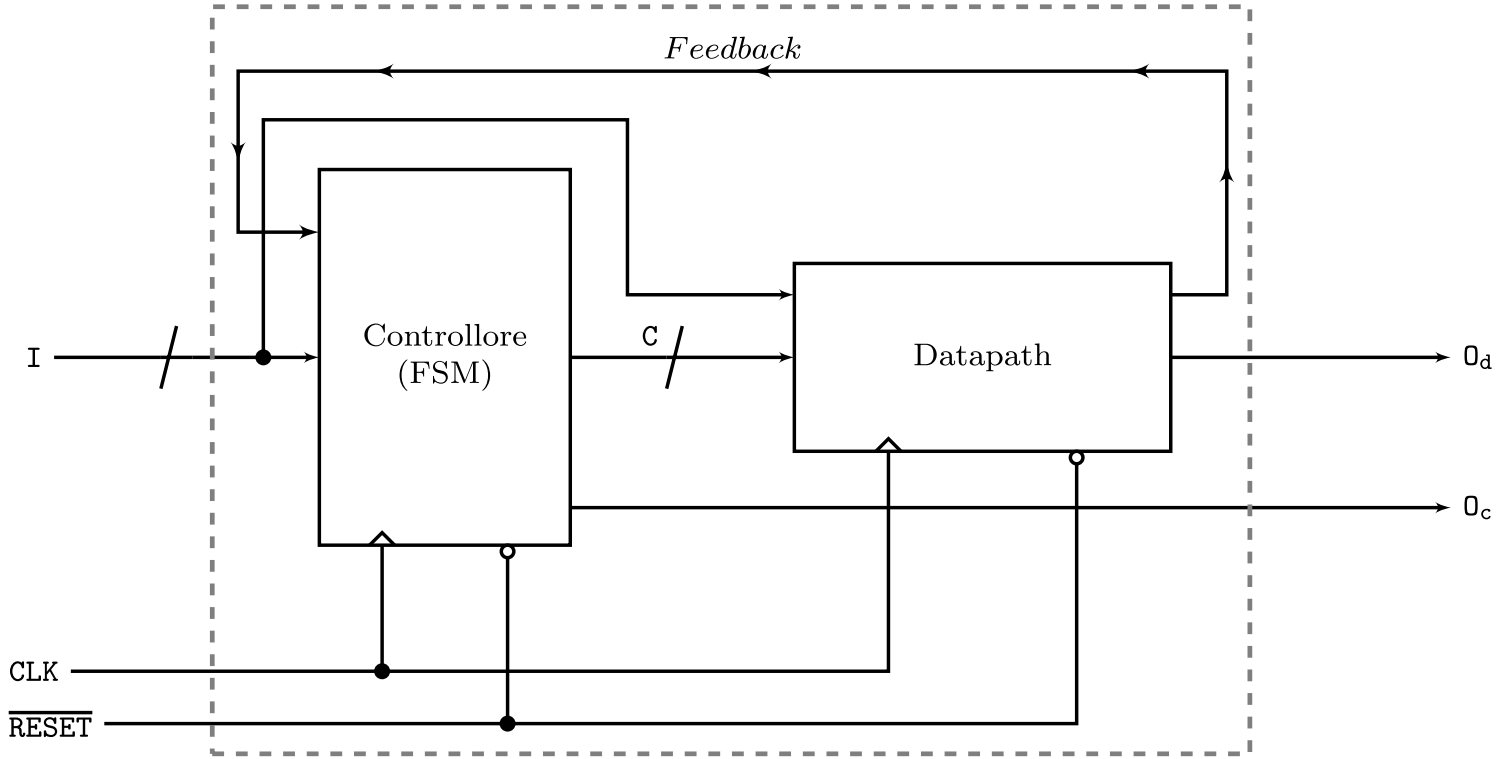
- Il *controllore* invia comandi e segnali al *datapath* senza alcuna retroazione
- Le *uscite* del sistema possono essere generate sia dal *controllore* sia dal *datapath*
- Gli *ingressi* del sistema possono interessare *entrambi* i componenti



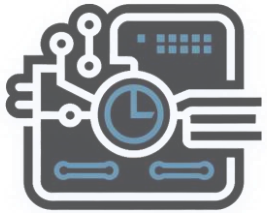
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Sistemi a controllo retroazionato

- Il controllore *riceve informazioni di ritorno* dal datapath
  - A fini di controllo dell'operato del sistema
  - Da usare come dato durante le successive fasi dell'algoritmo
- La *retroazione* è comunemente indicata con il termine «*feedback*»



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



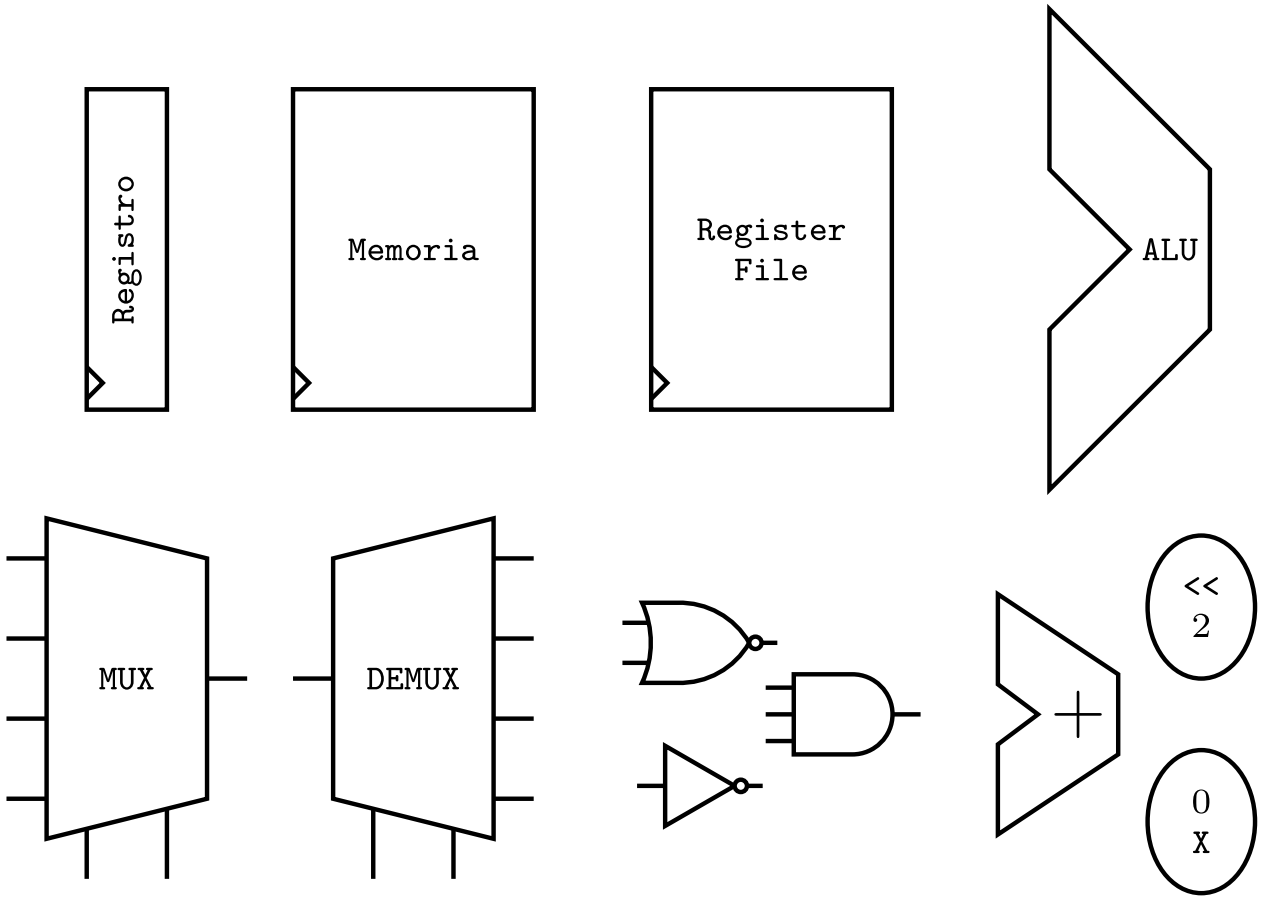
Design di sistemi con controllore e datapath

1. Descrivere le *funzionalità* del sistema da modellare
2. Determinare quali *componenti* del datapath siano necessari
3. Definire le *interconnessioni* tra elementi del datapath
4. Identificare i *segnali di input e output del controllore*
5. Definire la *sequenza di segnali* che il controllore deve generare perché il sistema presenti il *comportamento atteso*
6. Definire la *FSM del controllore* adatto allo scopo
7. *Verifica*, manuale o automatica, del sistema complessivo.
   In caso di errori e/o incongruenze *rivedere* i passi da (2) a seguire
8. *Implementazione e test* del sistema definitivo.
   In caso di errori e/o incongruenze *rivedere* i passi da (2) a seguire

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Componenti del datapath

- Componenti comuni che si possono solitamente trovare in nella parte datapath di un sistema sono registri, memorie, porte logiche, multiplexer, componenti aritmetici e così via…

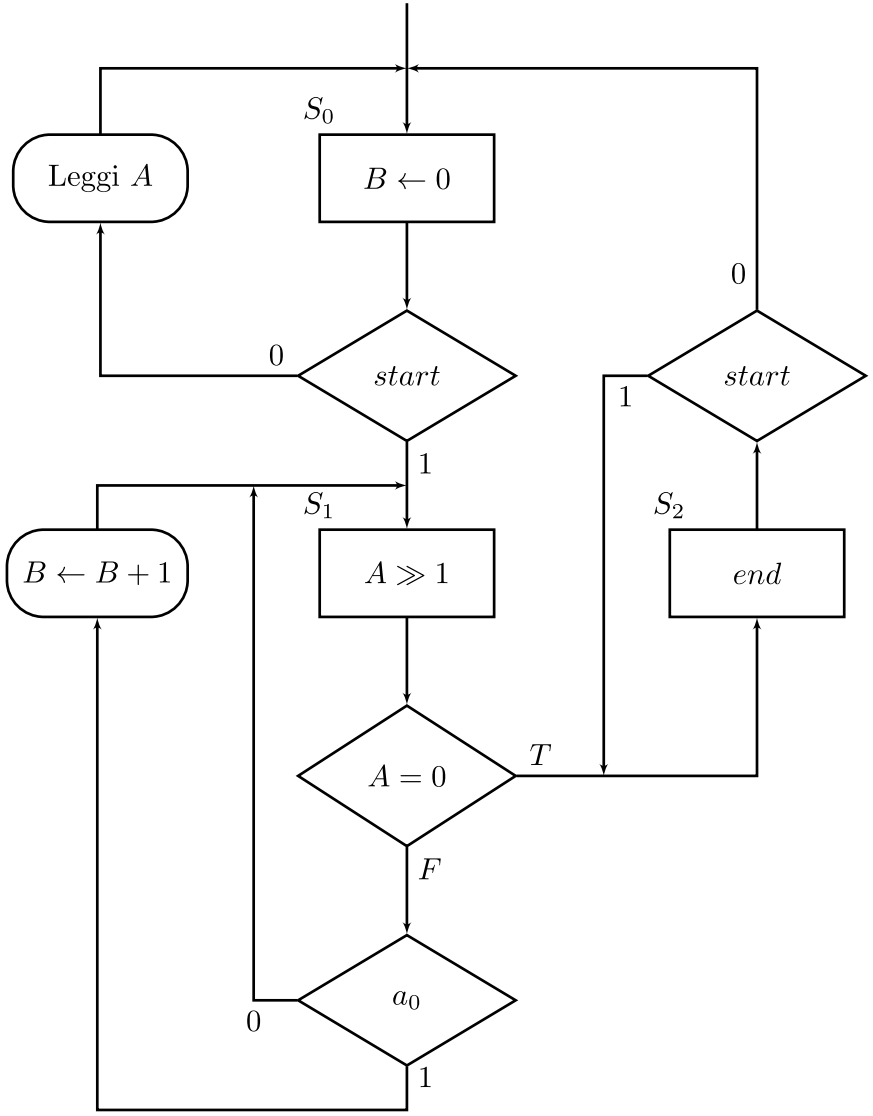


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 1
Bit counter

- A partire dalla specifica espressa in pseudocodice proposta a seguire, si vuole implementare il circuito corrispondente sfruttando il paradigma controller/datapath
- Si fa uso di una descrizione in formato ASM dell'algoritmo stesso, da usare come base per il resto del design

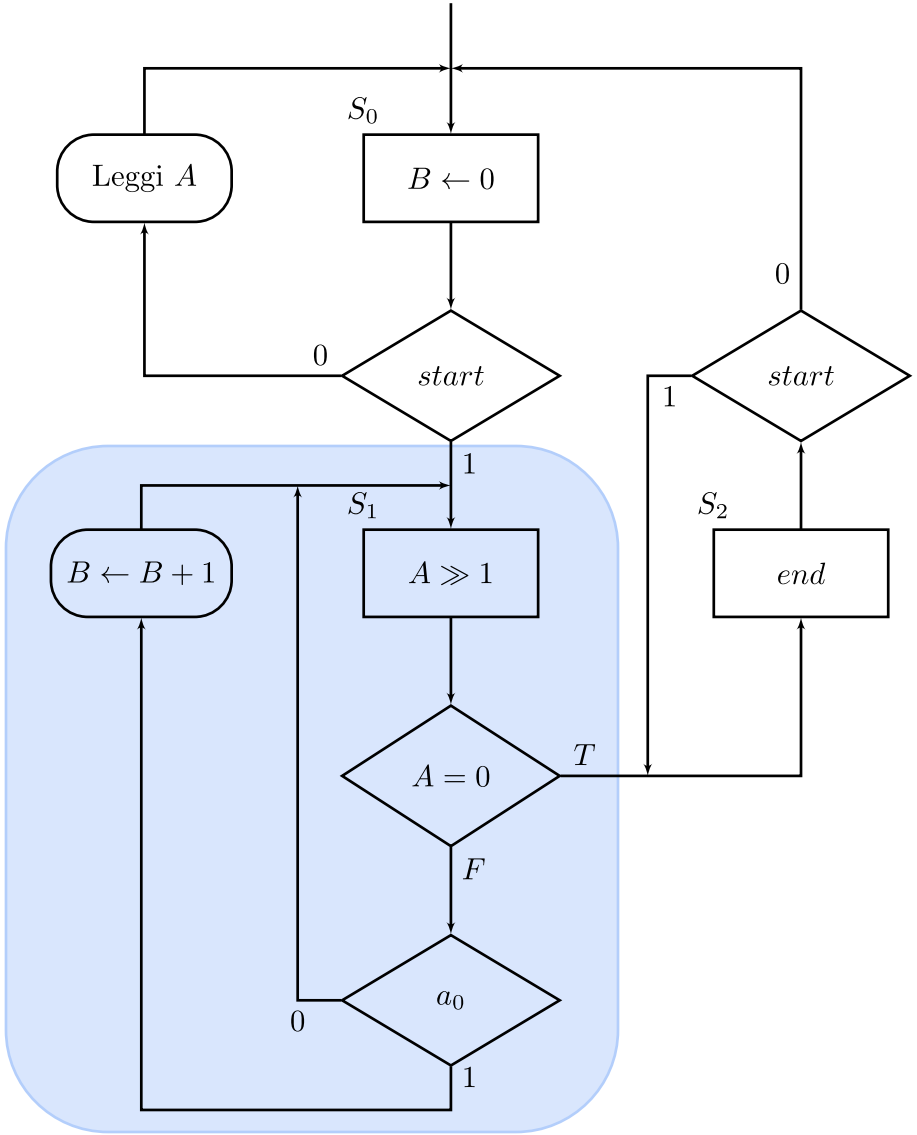
$$
\begin{aligned}
&B \leftarrow 0 \\
&\textbf{while } A \neq 0 \textbf{ do} \\
&\quad \textbf{if } a_0 = 1 \textbf{ then} \\
&\quad\quad B \leftarrow B + 1 \\
&\quad A \gg 1
\end{aligned}
$$



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 1
Bit counter

- È importante ricordare come le ASM rappresentino anche informazioni implicite sul timing del sistema
- La porzione evidenziata nell'immagine è interamente associata allo stato S_1 e i box di decisione relativi ai valori di A e a_0 tengono conto del valore di A allo stato corrente
- Il risultato dell'operazione di shift sarà «visibile» solo al colpo di clock successivo, quando il valore del registro verrà aggiornato

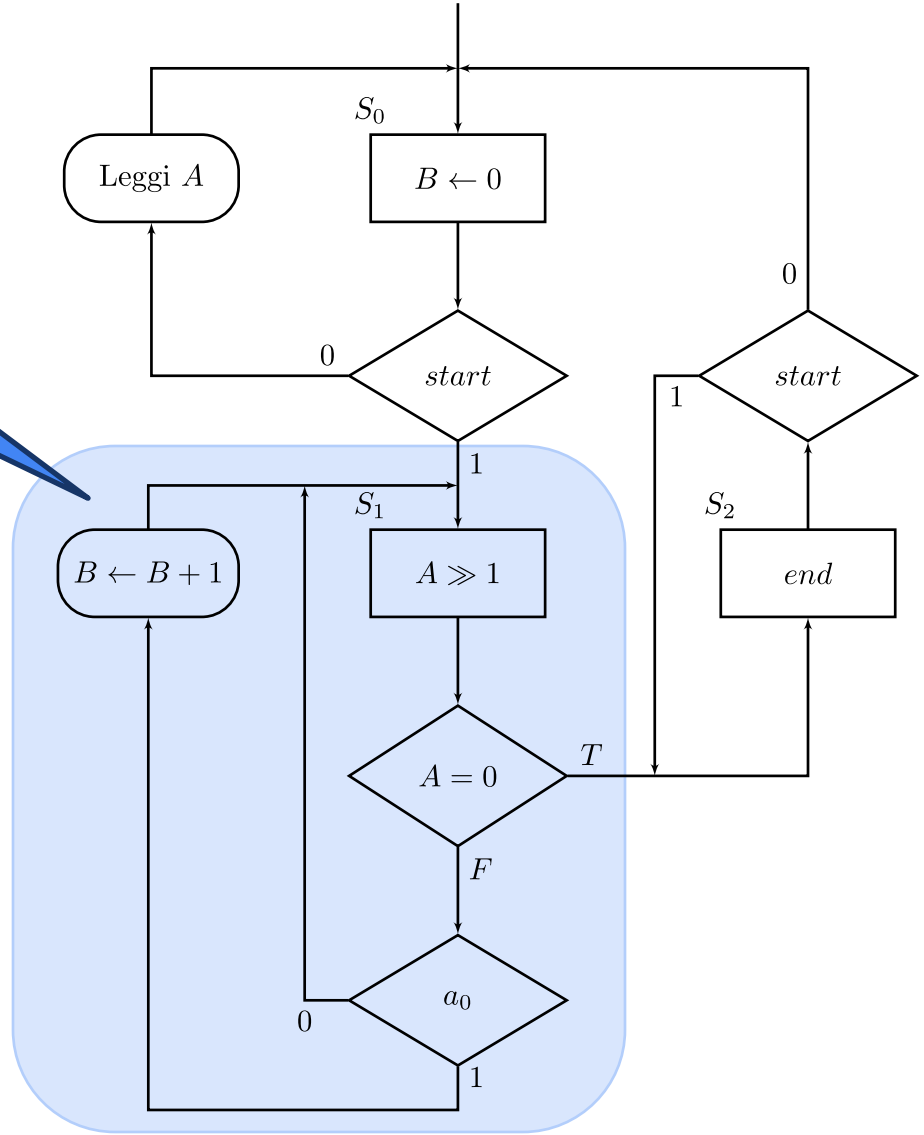


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 1
Bit counter

- È importante ricorda… rappresentino anche… implicite sul timing …
- La porzione evidenziata nell'immag… è interamente associata allo stato S_1 e i box di decisione relativi ai valori di A e a_0 tengono conto del valore di A allo stato corrente
- Il risultato dell'operazione di shift sarà «visibile» solo al colpo di clock successivo, quando il valore del registro verrà aggiornato

In questa sezione distinguiamo per il registro A il suo valore corrente A_{curr} e il suo valore futuro A_{next}



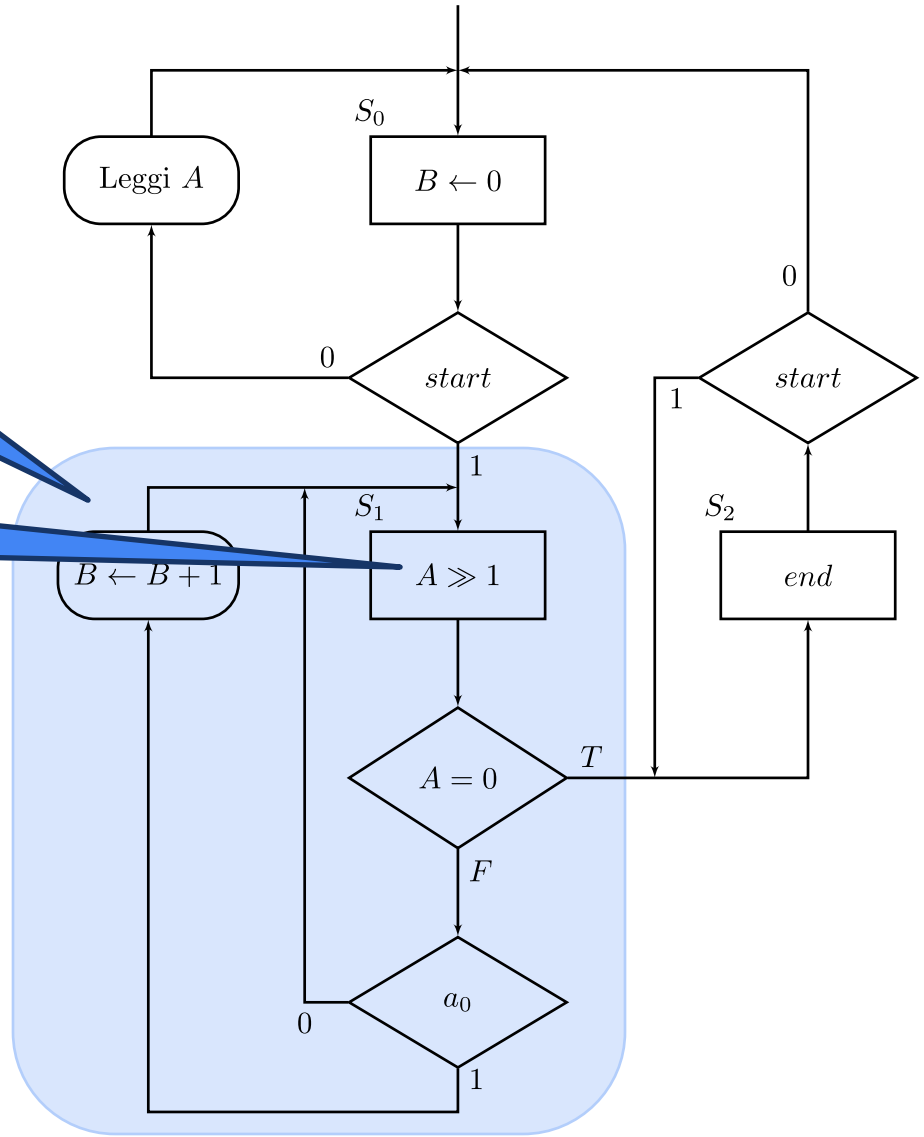
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 1
Bit counter

- È importante ricorda[illegible] rappresentino anche[illegible] implicite sul timing [illegible]
- La porzione evidenziata nell'immag[illegible]è interamente associata allo stato S_1 e i box di deci[illegible]i di A e a_0 tengono co[illegible]o stato corrente
- Il risultato dell'operazione di shift sarà «visibile» solo al colpo di clock successivo, quando il valore del registro verrà aggiornato

In questa sezione distinguiamo per il registro A il suo valore corrente A_{curr} e il suo valore futuro A_{next}

L'operazione di shift calcola il valore di A_{next} che sarà memorizzato e reso visibile nel registro A al successivo colpo di clock



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

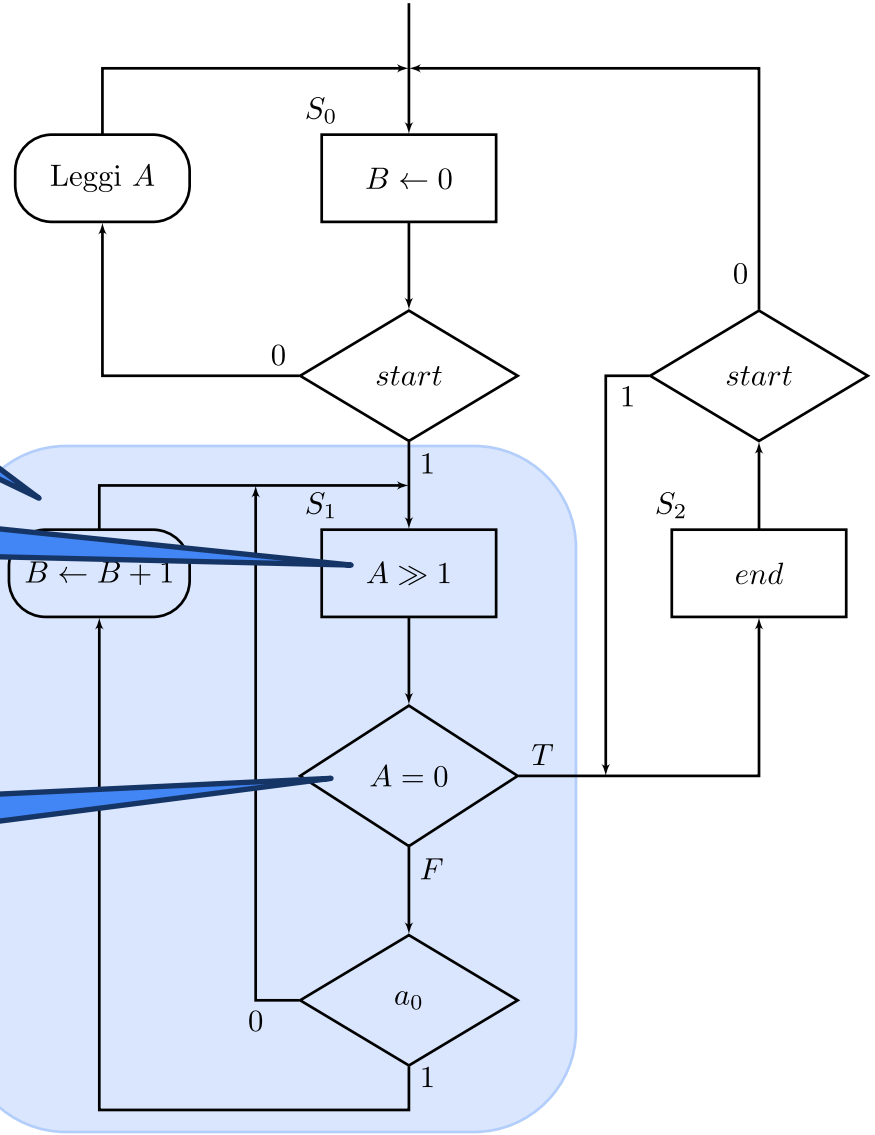
Esempio 1
Bit counter

- È importante ricorda… rappresentino anche… implicite sul timing …
- La porzione evidenziata nell'immag… è interamente associata allo stato S_1 e i box di deci… di A e a_0 tengono c… o stato corrente
- Il risultato dell'operazione di shift sarà «visibile» s… successivo … registro verrà aggio…

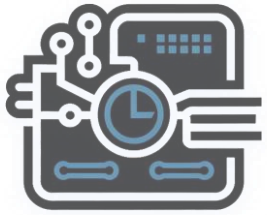
In questa sezione distinguiamo per il registro A il suo valore corrente A_{curr} e il suo valore futuro A_{next}

L'operazione di shift calcola il valore di A_{next} che sarà memorizzato e reso visibile nel registro A al successivo colpo di clock

Le condizioni su A (e a_0 di conseguenza) sono valutate sul valore corrente A_{curr} del registro

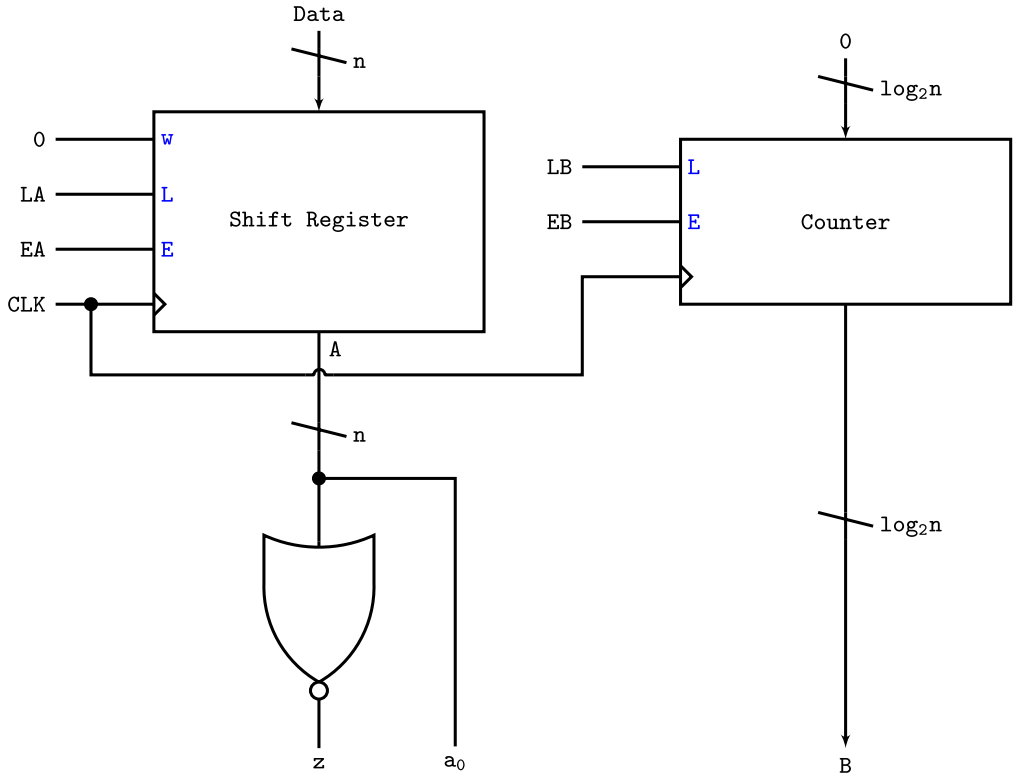


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Esempio 1
Bit counter

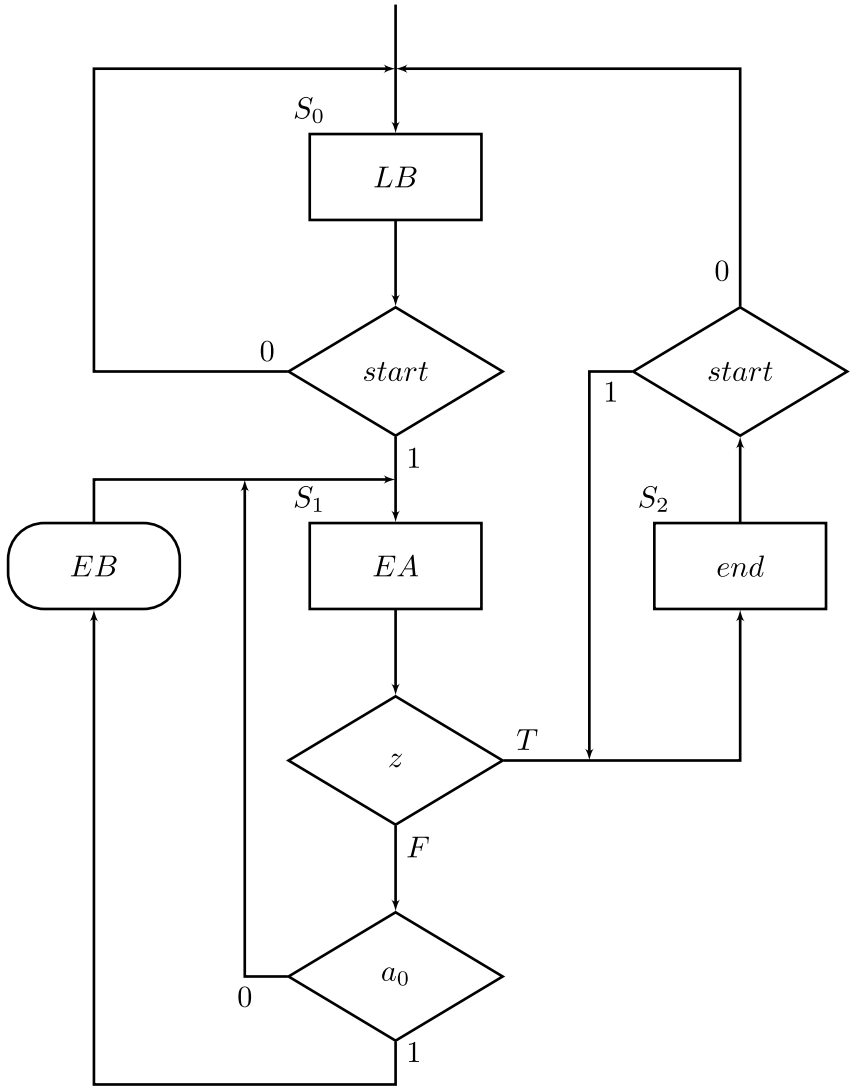
- Dall'analisi della ASM appena introdotta è possibile definire quali componenti siano necessari per realizzare il datapath
  - Assumiamo una rappresentazione dei dati su «n bit»
  - Uno shift register con scorrimento a sinistra, con possibilità di caricare dati in parallelo e segnale di abilitazione
  - Un contatore con possibilità di caricare dati in parallelo. Anziché fare affidamento a reset, si preferisce caricare esplicitamente il valore 0 nello stato iniziale
  - La porta logica NOR implementa la funzione di comparazione di A con il valore 0



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 1
Bit counter

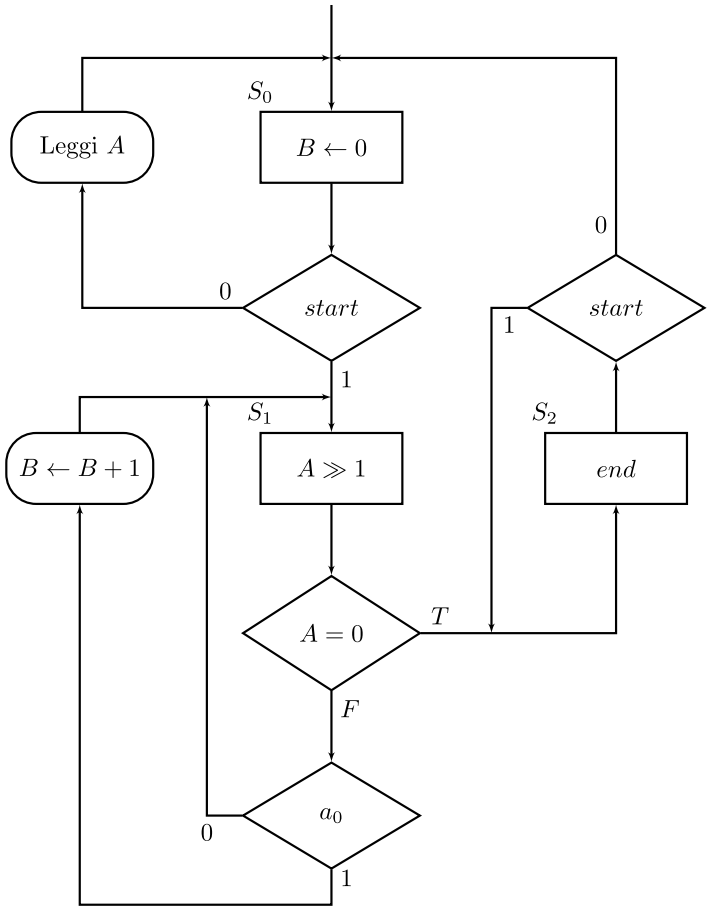
- Nella ASM che rappresenta il solo controllore sono riportati solo i segnali di controllo
- In input sono presenti i segnali $start$, a_0 e z
- In output sono generati i segnali EA, EB, LB ed end
  - Si assume che circuiteria esterna sia responsabile del caricamento e gestione del valore di A
  - Per compattezza, quando un segnale è asserito, appare solo il nome nell'elemento corrispondente della ASM

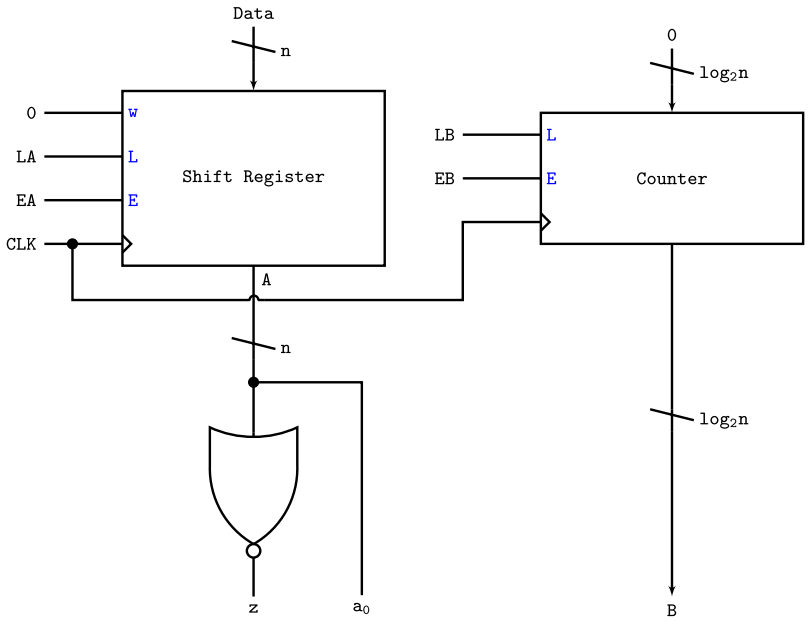


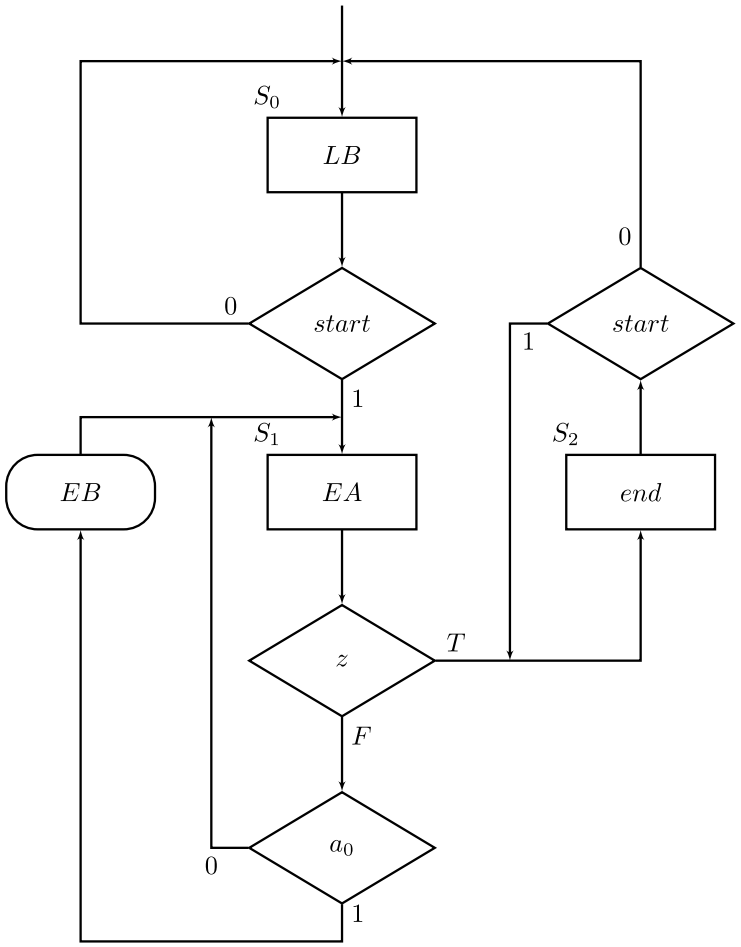
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 1
Bit counter

- Affiancando tutti «i pezzi» per avere una migliore visione di insieme…







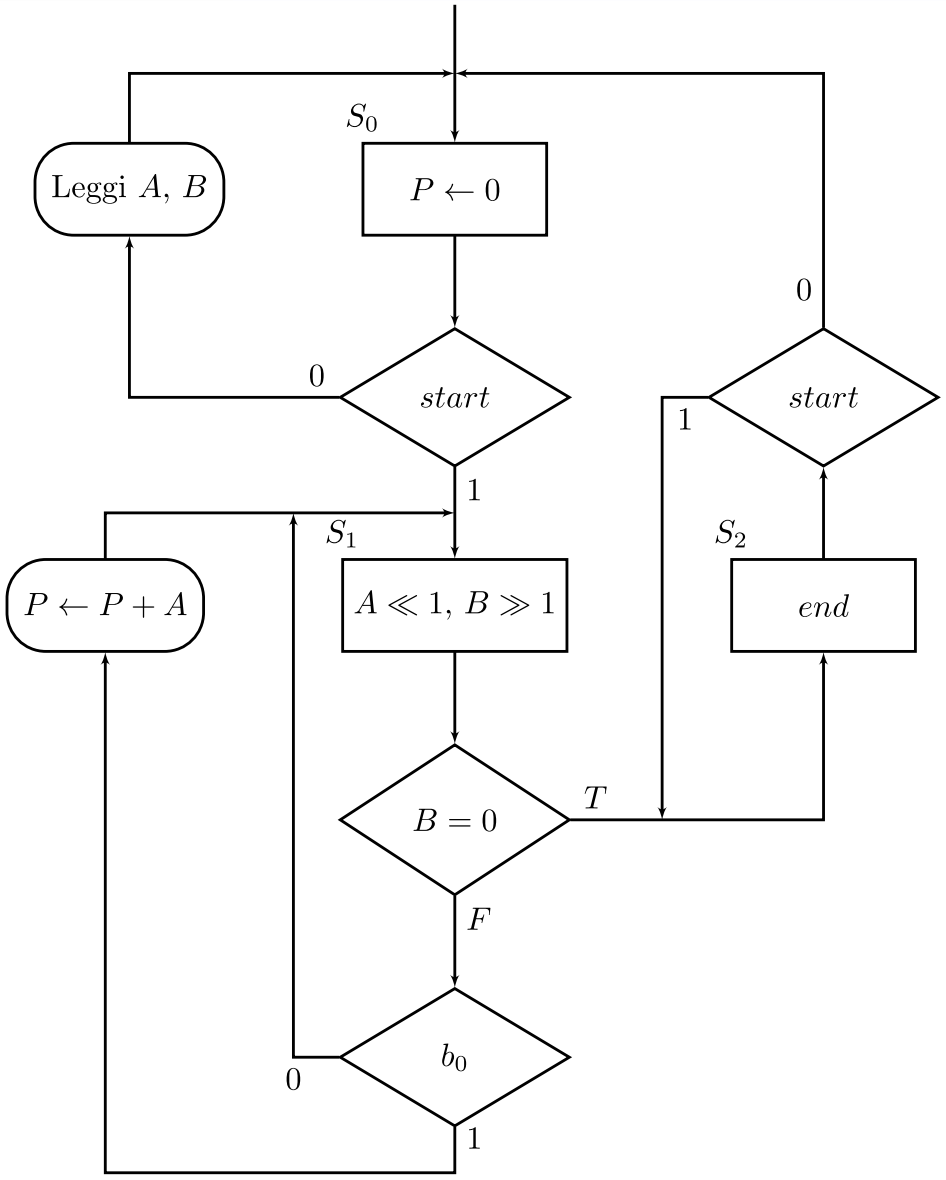
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 2
Moltiplicatore binario SHIFT-ADD

- A partire dalla specifica espressa in pseudocodice proposta a seguire, si vuole implementare il circuito corrispondente sfruttando il paradigma controller/datapth
- Si fa uso di una descrizione in formato ASM dell'algoritmo stesso, da usare come base per il resto del design

$P \leftarrow 0$
for $i \leftarrow 0$ to $n-1$ **do**
  if $b_i = 1$ **then**
    $P \leftarrow P + A$
  $A \ll 1$
  $B \gg 1$

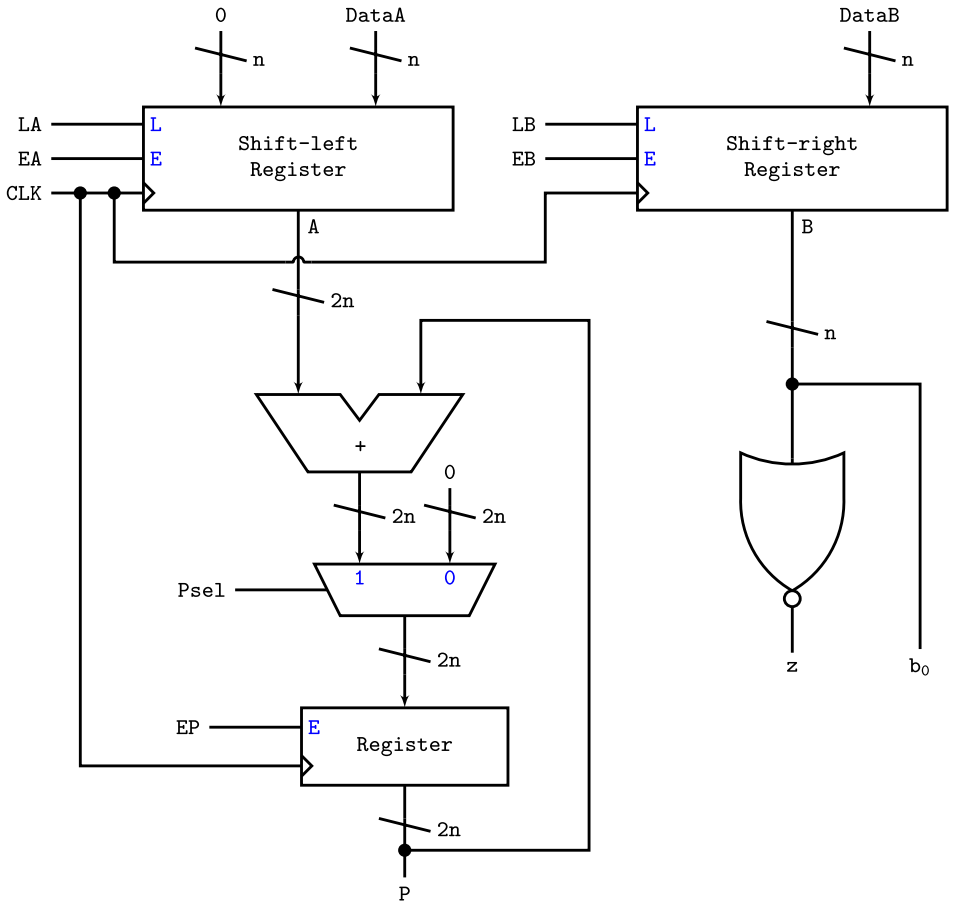
$$
\begin{array}{cccccccc}
 & & & & 1 & 1 & 0 & 1 \\
 & & & \times & 1 & 0 & 1 & 1 \\
\hline
 & & & & 1 & 1 & 0 & 1 \\
 & & & 1 & 1 & 0 & 1 & \\
 & & 0 & 0 & 0 & 0 & & \\
 & 1 & 1 & 0 & 1 & & & \\
\hline
1 & 0 & 0 & 0 & 1 & 1 & 1 & 1
\end{array}
$$



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 2
Moltiplicatore SHIFT-ADD

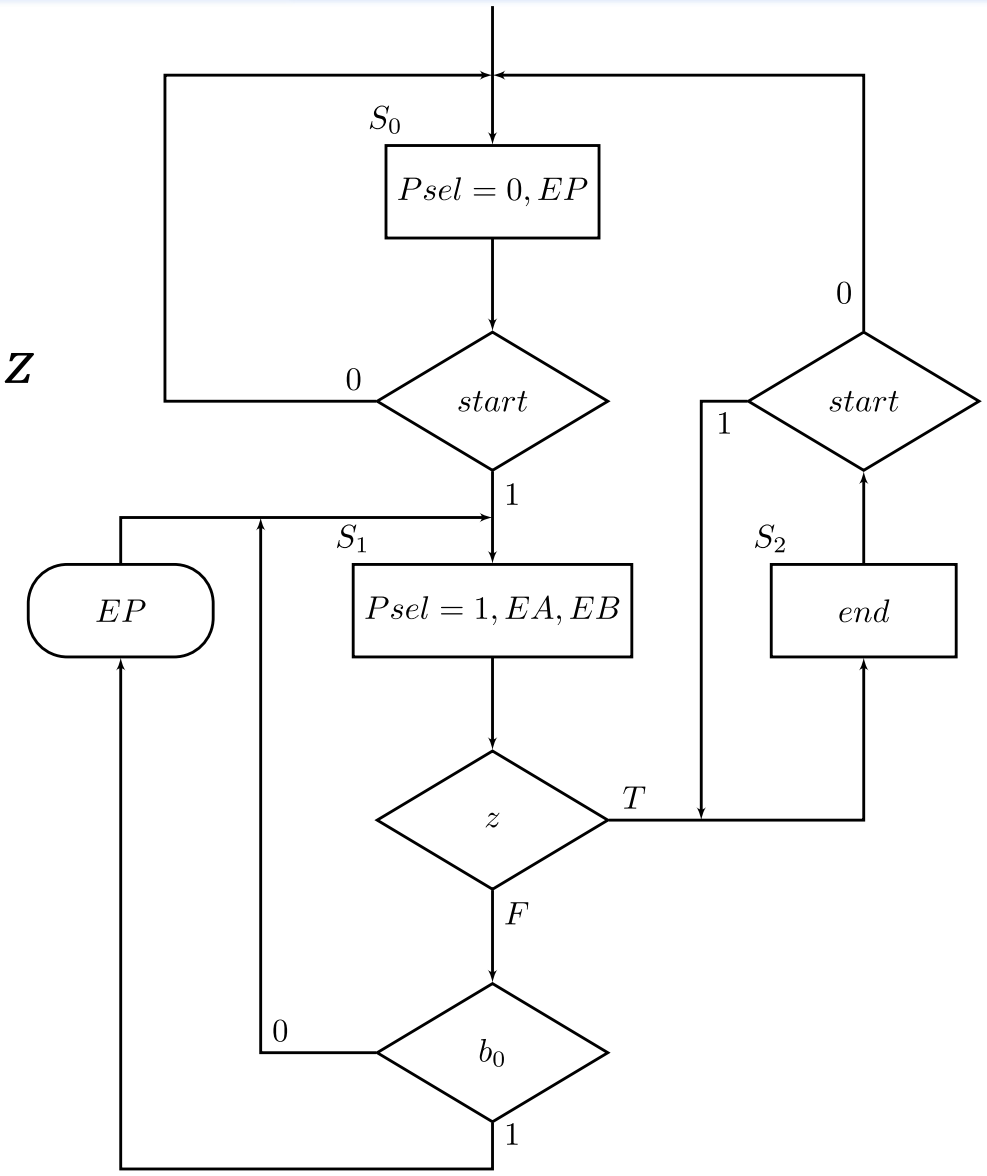
- Dall'analisi della ASM appena introdotta è possibile definire quali componenti siano necessari per realizzare il datapath
  - Due shift register con scorrimento a sx/dx, con possibilità di caricare dati in parallelo e segnale di abilitazione
  - Un registro con enable per il valore di P
  - Un sommatore per calcolare il valore $P + A$
  - Un multiplexer 2-a-1 per controllare i dati caricati nel registro dedicato a P
  - La porta logica NOR implementa la funzione di comparazione di B con il valore 0



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 2
Moltiplicatore SHIFT-ADD

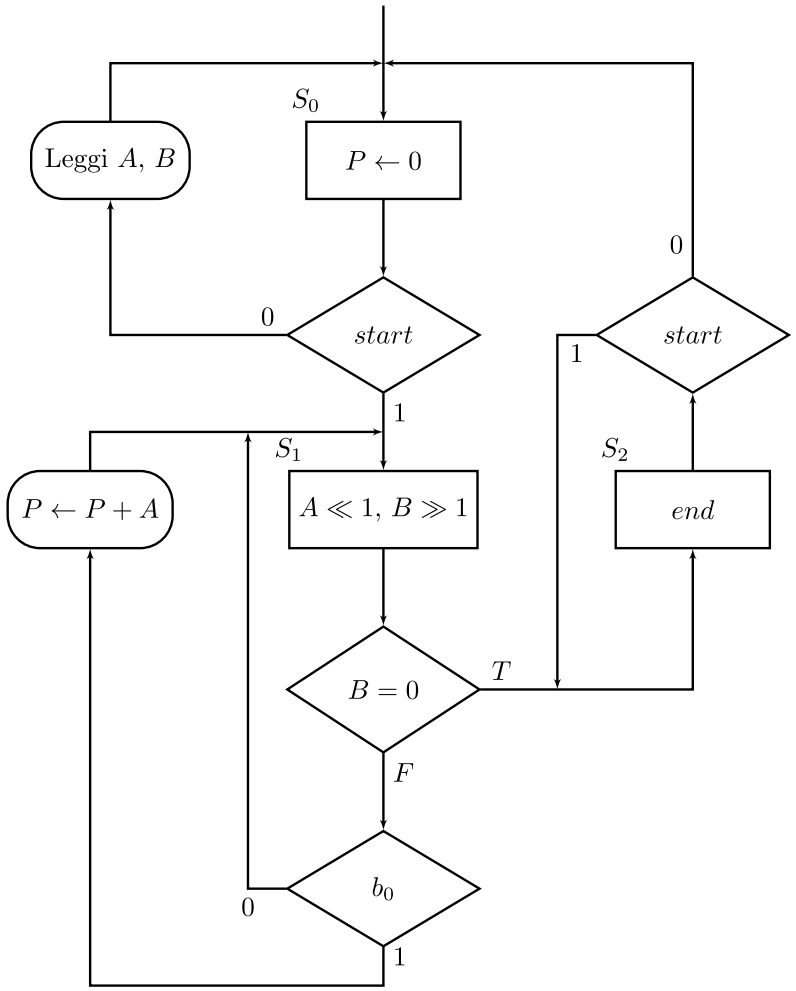
- Nella ASM che rappresenta il solo controllore sono riportati solo i segnali di controllo
- In input sono presenti i segnali *start*, *Psel* e z
- In output sono generati i segnali EA, EB, EP ed *end*
  - Si assume che circuiteria esterna sia responsabile del caricamento e gestione del valore di A e di B

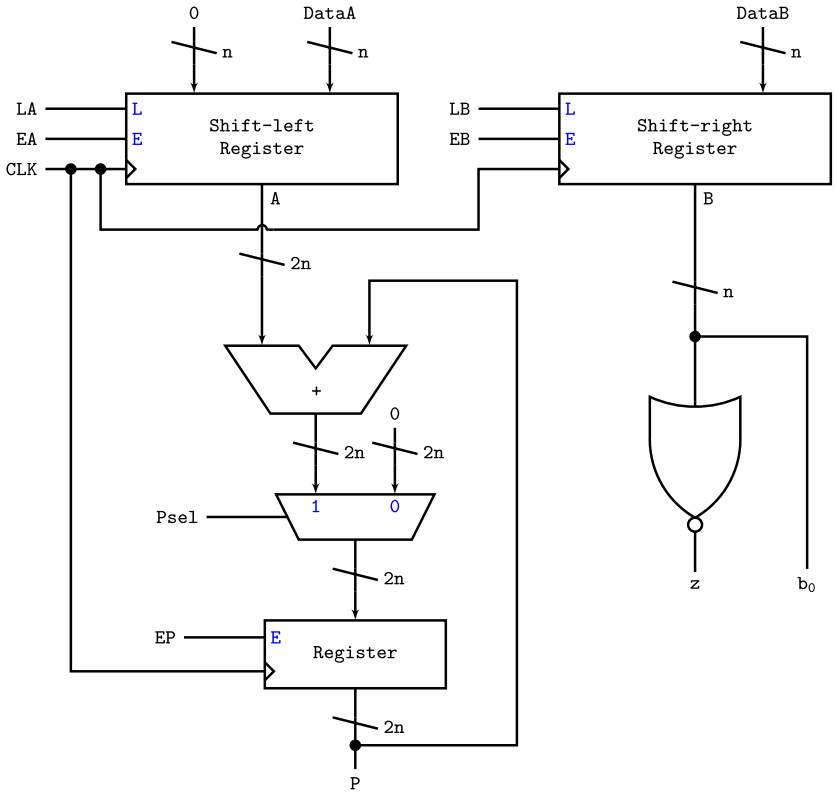


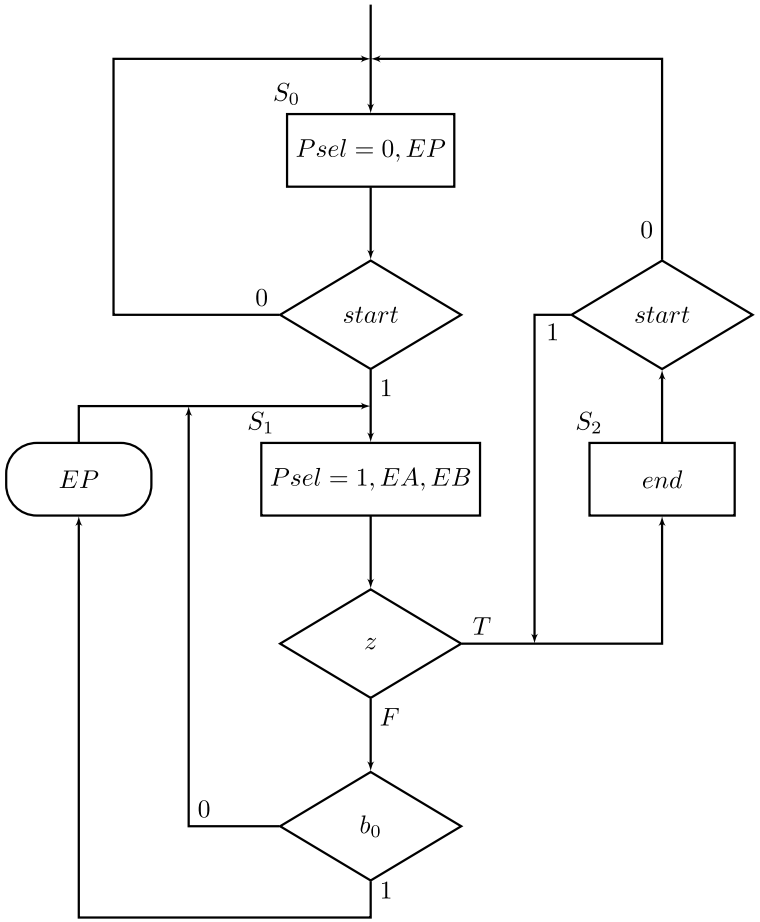
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 2
Moltiplicatore SHIFT-ADD

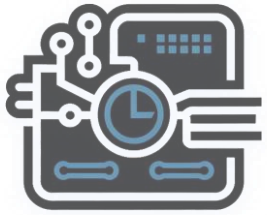
- Affiancando tutti «i pezzi» per avere una migliore visione di insieme…







2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Esempio 3
Divisione binaria «in colonna»

- Si vuole realizzare un sistema controller/datapath adatto a implementare l'operazione di divisione in colonna tra numeri binari, calcolando quoziente Q e resto R dati due valori A e B in input

$$
\begin{array}{rl}
 & 0\,1\,5 \\
\cline{2-2}
9 \,\big|\, 1\,4\,0 & 1 < 9 \\
-\;\;0 & \leftarrow 0 \cdot 9 \\
\cline{2-2}
\quad 1\,4 & 14 > 9 \\
-\;\;\;9 & \leftarrow 1 \cdot 9 \\
\cline{2-2}
\quad\;\; 5\,0 & 50 > 9 \\
-\;\;4\,5 & \leftarrow 5 \cdot 9 \\
\cline{2-2}
\quad\;\; 5 &
\end{array}
$$

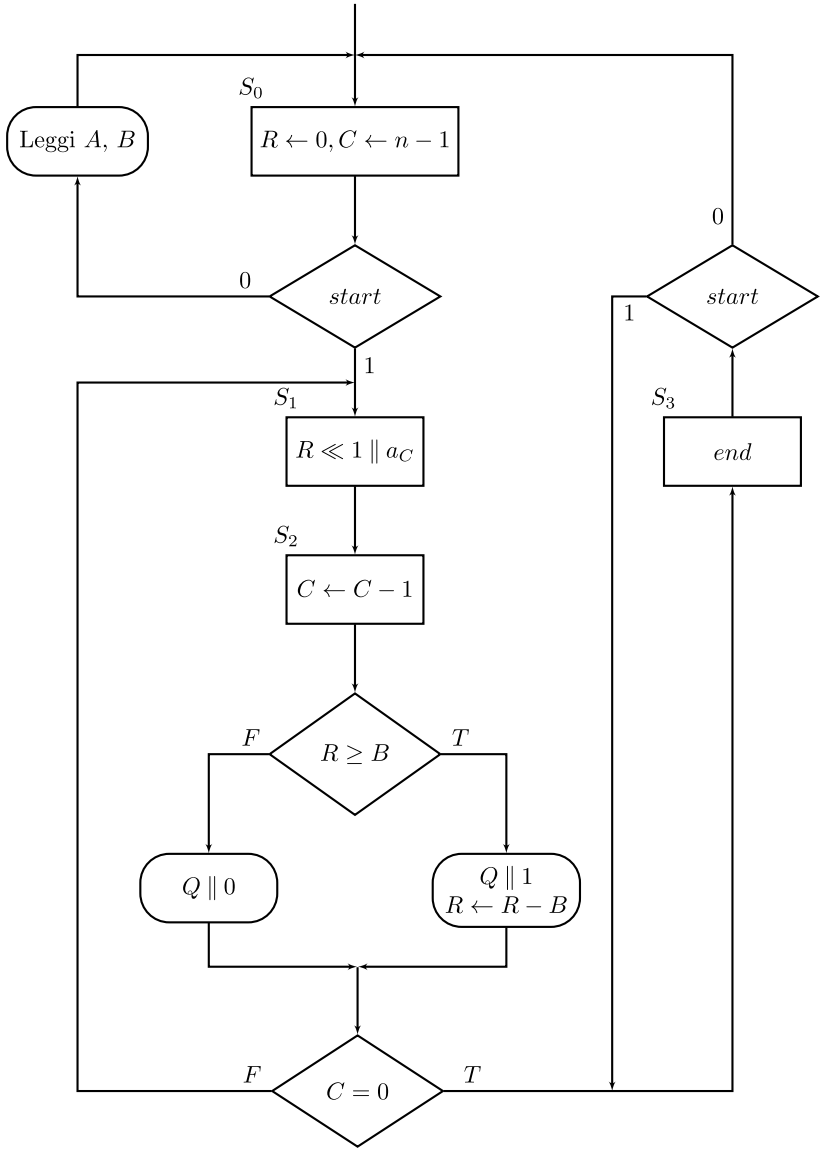
$$
\begin{array}{l}
R \leftarrow 0 \\
\textbf{for } i \leftarrow n-1 \text{ to } 0 \textbf{ do} \\
\quad R \ll 1 \parallel a_i \\
\quad \textbf{if } R \geq B \textbf{ then} \\
\quad\quad q_i \leftarrow 1 \\
\quad\quad R \leftarrow R - B \\
\quad \textbf{else} \\
\quad\quad q_i \leftarrow 0
\end{array}
$$

$$
\begin{array}{rl}
 & 0\,0\,0\,0\,1\,1\,1\,1 \\
\cline{2-2}
1\,0\,0\,1 \,\big|\, 1\,0\,0\,0\,1\,1\,0\,0 & 1 < 1001 \\
-\;\;0 & \leftarrow 0 \cdot 1001 \\
\cline{2-2}
\quad 1\,0 & 10 < 1001 \\
-\;\;\;0 & \leftarrow 0 \cdot 1001 \\
\cline{2-2}
\quad 1\,0\,0 & 100 < 1001 \\
-\;\;\;\;0 & \leftarrow 0 \cdot 1001 \\
\cline{2-2}
\quad 1\,0\,0\,0 & 1000 < 1001 \\
-\;\;\;\;\;0 & \leftarrow 0 \cdot 1001 \\
\cline{2-2}
\quad 1\,0\,0\,0\,1 & 10001 > 1001 \\
-\;\;\;1\,0\,0\,1 & \leftarrow 1 \cdot 1001 \\
\cline{2-2}
\quad\;\; 1\,0\,0\,0\,1 & 10001 > 1001 \\
-\;\;\;\;1\,0\,0\,1 & \leftarrow 1 \cdot 1001 \\
\cline{2-2}
\quad\;\;\; 1\,0\,0\,0\,0 & 10000 > 1001 \\
-\;\;\;\;\;1\,0\,0\,1 & \leftarrow 1 \cdot 1001 \\
\cline{2-2}
\quad\;\;\;\;\; 1\,1\,1\,0 & 1110 > 1001 \\
-\;\;\;\;\;\;1\,0\,0\,1 & \leftarrow 1 \cdot 1001 \\
\cline{2-2}
\quad\;\;\;\;\;\;\; 1\,0\,1 &
\end{array}
$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 3
Divisione binaria «in colonna»

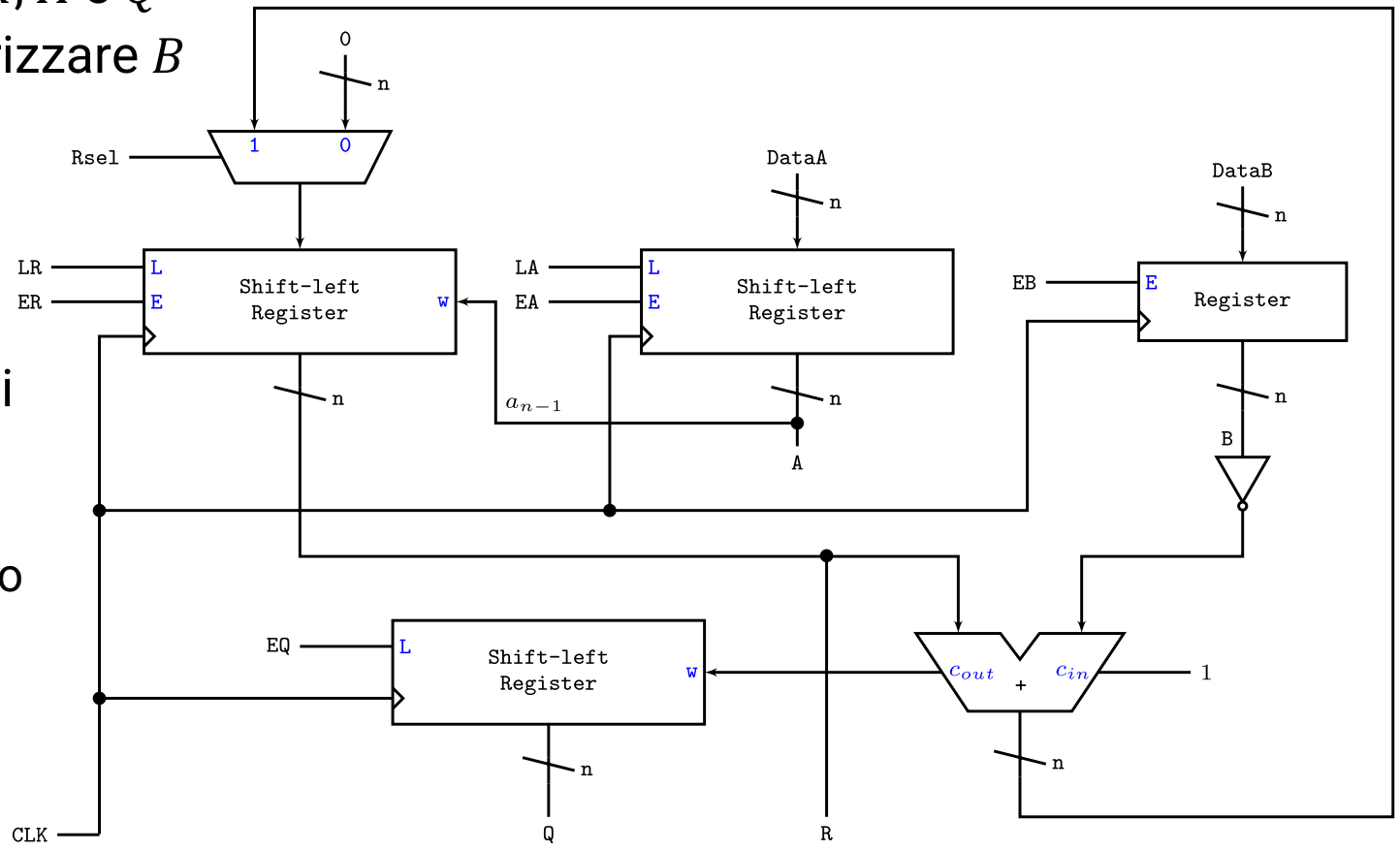
- Come in precedenza, si parte da una rappresentazione mediante ASM
- Il segnale C è usato per modellare il contatore del ciclo ***for***



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 3
Divisione binaria «in colonna»

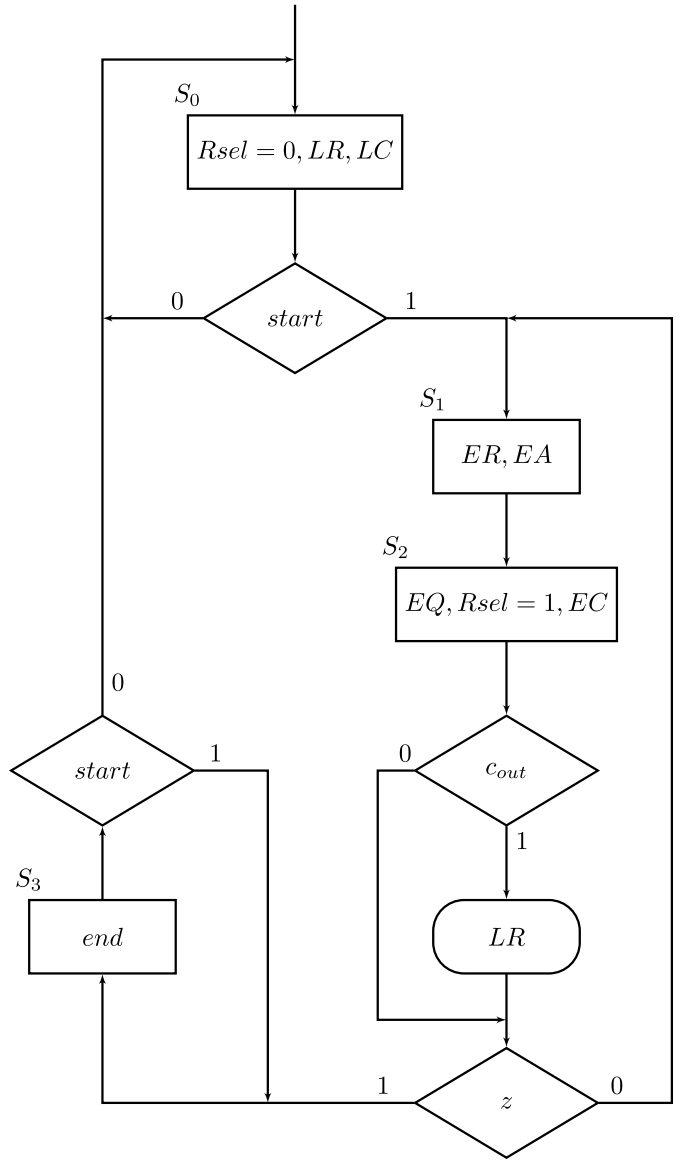
- Per realizzare il datapath in oggetto servono
  - Uno shift register ciascuno per R, A e Q
  - Un registro semplice per memorizzare B
  - Un sottrattore per realizzare la differenza tra R e B
  - Un multiplexer 2-a-1 per il caricamento (o il reset) di R
  - Per compattezza si sono omessi
    - Il registro dedicato al counter
    - La porta NOR per verificare l'estremo inferiore del conteggio



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 3
Divisione binaria «in colonna»

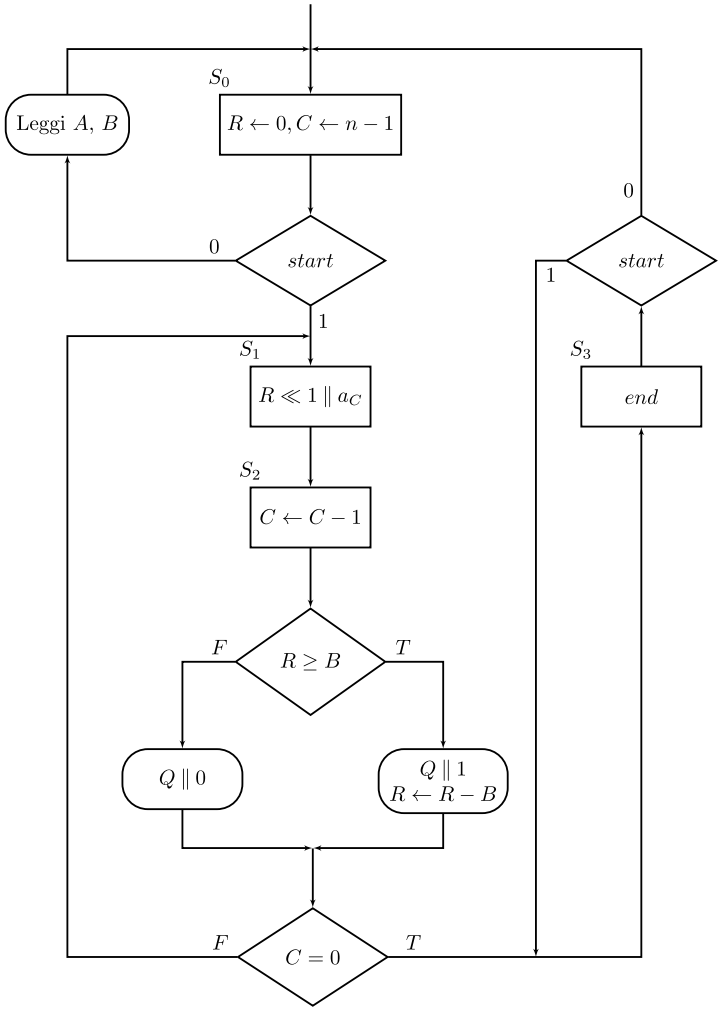
- Nella ASM che rappresenta il solo controllore sono riportati solo i segnali di controllo
- Nello stato S_2 il valore di c_{out} implicitamente determina se occorra caricare il risultato del sommatore in R
- Nello stesso stato è abilitato anche il segnale di abilitazione per il registro che memorizza Q
- Non serve specificare quale bit shiftare in ingresso, poiché c_{out} è direttamente collegato al relativo input seriale

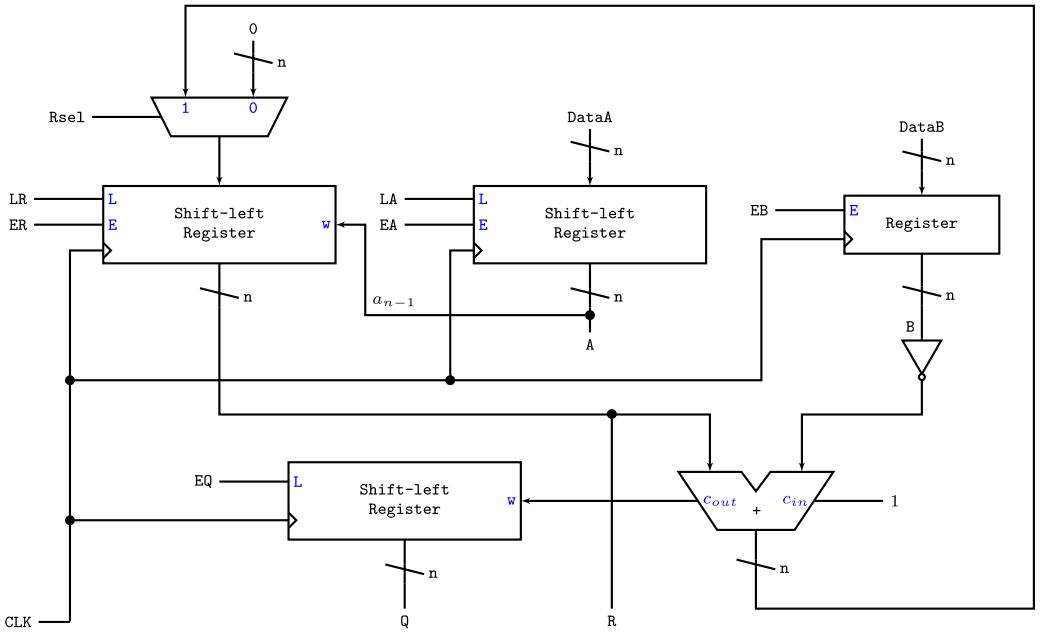


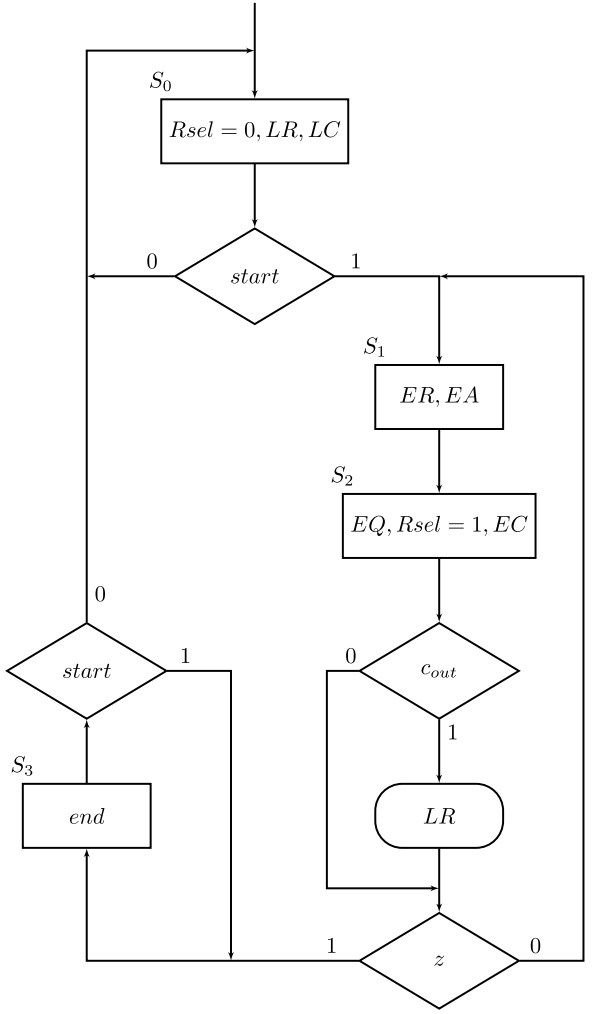
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 3
Divisione binaria «in colonna»

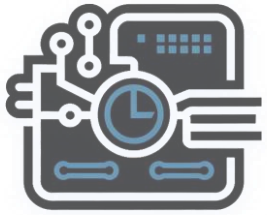
- Affiancando tutti «i pezzi» per avere una migliore visione di insieme…







2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

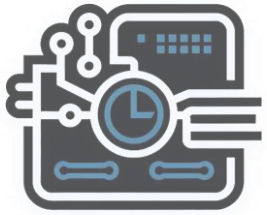


Esempio 4
Divisione binaria «in colonna» V2

A3

- Poiché gli stati S_1 ed S_2 sono in sequenza, servono 2 cicli di clock per iterazione
  - Riuscendo a fonderli, sarebbe sufficiente un singolo ciclo di clock per iterazione
- Nei due stati avvengono le seguenti operazioni:
  - In S_2 si carica in R il valore della sottrazione, se $c_{out} = 1$. R è sovrascritto.
  - In S_1, R viene sempre shiftato a sinistra e concatenato al MSB di A
- Per fondere i due stati servirebbe poter fare le due operazioni in contemporanea
  - Memorizzare la somma nella parte più significativa di R
  - Shiftare il MSB di A nel LSB di R senza che la memorizzazione della differenza lo sovrascriva
- Siccome la differenza riscrive completamente R, è necessario usare un flip-flop rr_0, separato da R per memorizzarne l'LSB, proveniente da A
- Nel «nuovo» stato S_1, la gestione di R ed rr_0 è:
  - Se $c_{out} = 0$: shift a sinistra di R e impostare il suo LSB al valore corrente di rr_0
  - Se $c_{out} = 1$: caricamento in parallelo in R della differenza
  - Shiftare l'MSB di A in rr_0 che funge da LSB di R nel confronto con B alla prossima iterazione

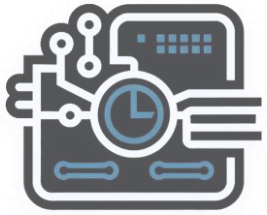
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Register Transfer Level (RTL)

- In precedenza abbiamo introdotto due grandi categorie di circuiti
  - Circuito combinatori
  - Circuiti sequenziali
- Un sistema digitale può essere visto come un sistema sequenziale composto da porte logiche, elementi di memoria e così via
  - Finché sono relativamente semplici, è possibile descriverne e analizzarne il comportamento mediante le tabelle di verità
- Per gestire sistemi complessi, si preferisce adottare una visione modulare degli stessi
  - Sistema partizionato in più sotto-sistemi, detti moduli, ognuno con uno specifico scopo
  - Moduli realizzati usando registri, multiplexer, decoder, ALU, logica di controllo ecc…
  - Moduli interconnessi sfruttando data path e opportuni segnali di controllo

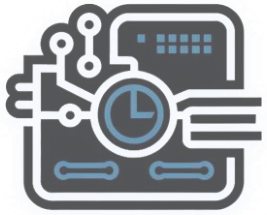
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Register Transfer Level (RTL)

- Un sistema digitale viene «reinterpretato» e definito sulla base dei registri che lo compongono e le operazioni che sono svolte sulle informazioni (binarie) memorizzate
- Le operazioni prese in considerazioni sono, ad esempio, caricamenti in memoria, shift, conteggi, …
- Il flusso di informazioni tra elementi del sistema e il processamento dei dati memorizzati nei registri sono comunemente definiti con il termine «Register Transfer Operations»
- Complessivamente, un sistema è descritto in RTL sulla base di:
  - Insieme dei registri del sistema
  - Operazioni effettuate sui dati memorizzati nei registri
  - Funzionalità di controllo che supervisiona la sequenza di operazioni nel sistema

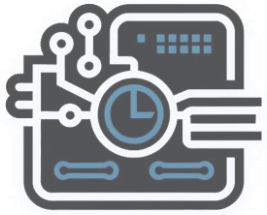
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Register Transfer Level (RTL)

- RTL è un'astrazione per definire le componenti digitali di un design
- La progettazione RTL solitamente prevede l’utilizzo di un linguaggio di descrizione dell’hardware hardware (HDL) come Verilog o VHDL
  - Questi linguaggi sono in grado di definire sistemi ad alti livelli di astrazione, ma generalmente è la semantica RTL a rappresentarne il sottoinsieme sintetizzabile
  - I costrutti del linguaggio possono essere usati in un flusso di sintesi logica che a sua volta crea l'astrazione a livello di gate del design, per essere poi utilizzata per le successive operazioni di implementazione
- RTL si basa su logica sincrona e include tre elementi principali
  - I registri che contengono le informazioni sullo stato
  - La logica combinatoria che definisce l’evoluzione del sistema
  - I clock che controllano quando lo stato cambia

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



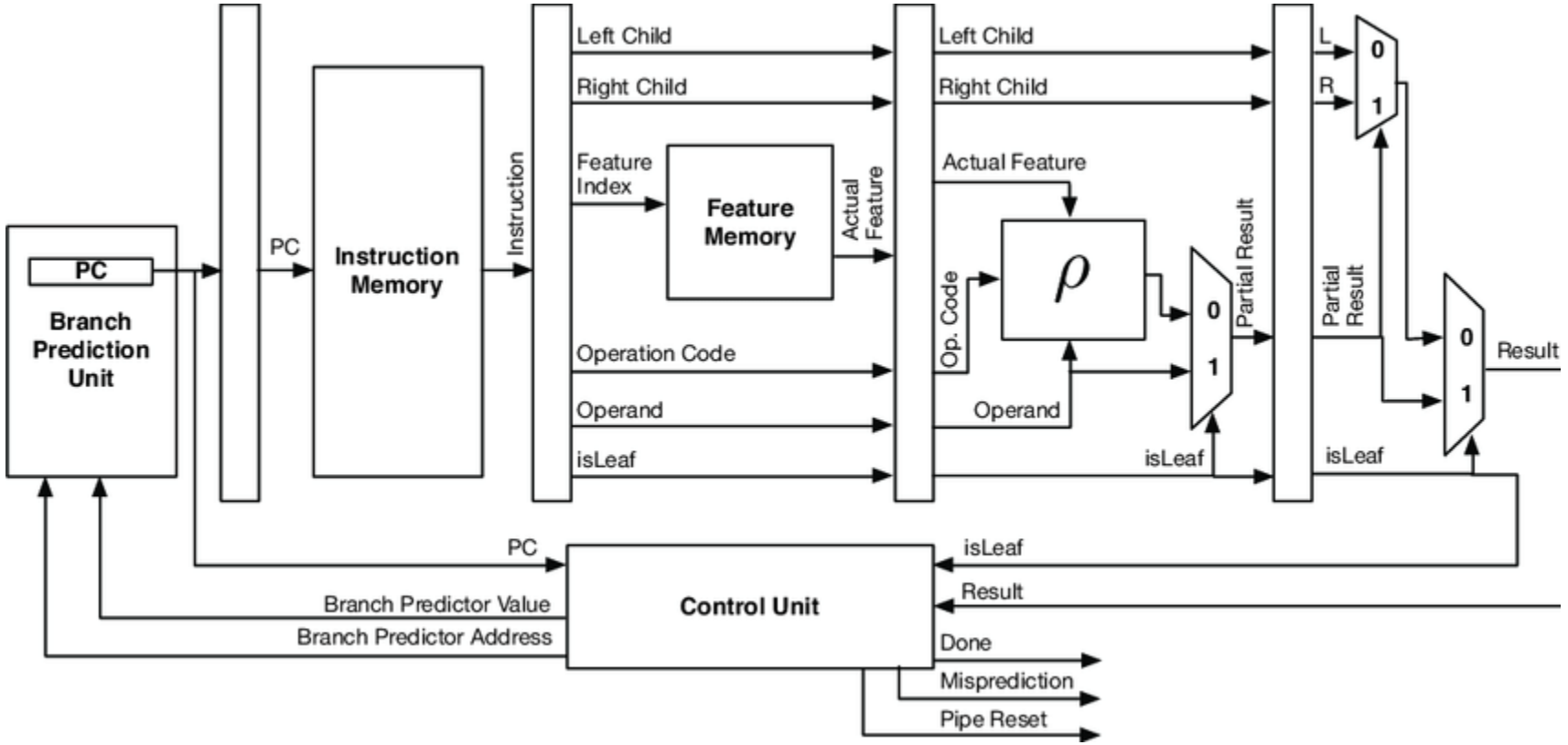
Register Transfer Level (RTL)
Esempio

- Consideriamo una casistica semplice: $R_2 \leftarrow R_1$
  - Denota il trasferimento dei contenuti di R_1 in R_2
  - Si tratta di un'operazione di copia: i contenuti del registro sorgente rimangono inalterati, mentre quelli del registro di destinazione sono sostituiti
  - Un'operazione che rappresenti un trasferimento tra registri fa supporre l'esistenza di un datapath che li colleghi
  - In generale, ci si aspetta che una certa operazione sia eseguita solo in specifiche situazioni, e in caso di sistemi sincroni, in corrispondenza ad un certo segnale di temporizzazione
    - $\text{if}(<\text{cond}>)\ \text{then}\ R_2 \leftarrow R_1$
    - L'eventuale ruolo del clock è sottointeso, e non appare esplicitamente nello statement
    - Nel caso la condizione divenisse vera prima della transizione del clock, l'operazione viene comunque svolta solamente in corrispondenza della transizione stessa
- Le quattro tipologie principali di operazioni RTL sono:
  - Trasferimenti, operazioni aritmetiche, operazioni logiche e shift

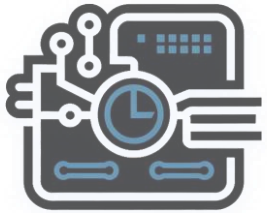
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

TVPU – Schema RTL

- A titolo di esempio, si propone lo schema RTL di una unità di processamento dedicata all'esplorazione e gestione di «*Decision Trees*»



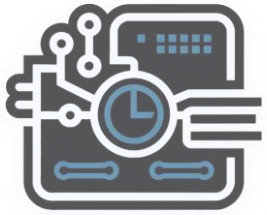
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Argomenti di approfondimento individuale

- A1) Somma e sottrazione in M&S/CA1/CA2
- A2) Circuito di rilevazione dell'overflow in CA2
- A3) Divisione binaria (Vers. 2)

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Somma e sottrazione in modulo e segno

A1

- Quando i segni sono uguali, basta sommare direttamente i due operandi. Il segno del risultato è uguale a quelli degli operandi stessi

4	0100		-4	1100	
+ 3	0011		+ (-3)	1011	
7	0111		-7	1111	

- Quando i segni sono diversi, bisogna sottrarre i due operandi ed il segno del risultato dipende da quello dell'operando con il modulo più grande

4	0100		-4	1100	
- 3	1011		+ 3	0011	
1	0001		-1	1001	

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Somma e sottrazione in CA1

- Si eseguono i calcoli interamente in CA1 con un eventuale ulteriore passaggio di «riporto all'inizio»

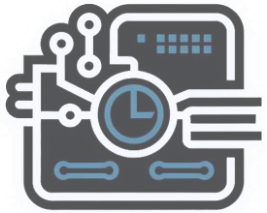
4	0100
+ 3	0011
7	0111

-4	1011
+ (-3)	1100
-7	10111
	→ 1
	1000

4	0100
+ (-3)	1100
1	10000
	→ 1
	0001

-4	1011
+ 3	0011
-1	1110

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Somma e sottrazione in CA1

- Il «riporto all'inizio» funziona poiché concettualmente equivale a sottrarre 2^n e sommare 1

- Se M > N

$$M - N = M + \bar{N} = M + (2^n - 1 - N) = (M - N) + 2^n - 1$$

- Se M+N < 2^n-1

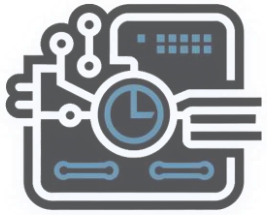
$$-M + (-N) = \bar{M} + \bar{N} = (2^n - M - 1) + (2^n - N - 1) = 2^n + [2^n - 1 - (M + N)] - 1$$

e dopo il riporto all'inizio

$$2^n - 1 - (M + N)$$

che è la rappresentazione di −(M+N) in CA1

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

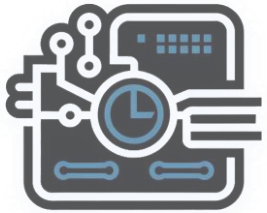


Somma e sottrazione in CA2

- Se il riporto sul segno è uguale al riporto in uscita, lo si ignora
- Se il riporto sul segno è diverso dal riporto in uscita, si ha una condizione di overflow
- Per via della maggior semplicità nello svolgere le somme, il CA2 è la rappresentazione più usata nei calcolatori

4	0100	-4	1100	4	0100	-4	1100
+ 3	0011	+ (-3)	1101	- 3	1101	+ 3	0011
7	0111	-7	11001	1	10001	-1	1111

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Somma e sottrazione in CA2

- Perché si può ignorare il riporto in uscita senza incorrere in errori?

- -M+N se N > M

$$-M + N = \bar{M} + N = (2^n - M) + N = 2^n + (N - M)$$

  ignorare il riporto in uscita è come sottrarre 2^n

- -M+N se M+N $\leq 2^n-1$

$$-M + (-N) = \bar{M} + \bar{N} = (2^n - M) + (2^n - N) = 2^n - (M + N) + 2^n$$

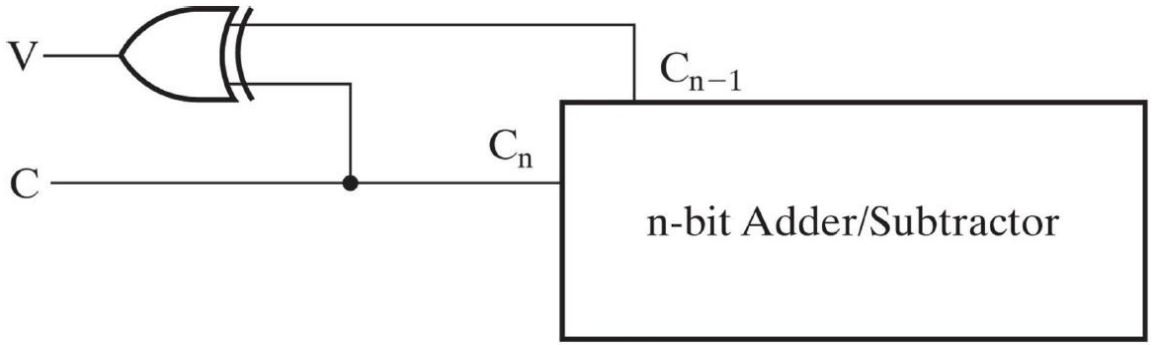
  che, ignorando il riporto, è la rappresentazione di −(M+N) in CA2

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

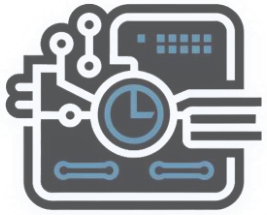
Circuito per la rilevazione dell'overflow

A2

- Numeri senza segno
  - C = 0
    - assenza di overflow nella somma
    - necessaria correzione del segno per la differenza
  - C = 1
    - overflow nella somma
    - segno corretto nella differenza
- Numeri con segno
  - V = 0
    - risultato corretto
  - V = 1
    - Overflow: valori diversi per il riporto sull'uscita e sul segno



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Esempio 4 – Divisione binaria «in colonna» V2

A3

- Poiché gli stati S_1 e S_2 sono in sequenza, servono 2 cicli di clock per iterazione
  - Riuscendo a fonderli, sarebbe sufficiente un singolo ciclo di clock per iterazione
- Nei due stati avvengono le seguenti operazioni:
  - In S_2 si carica in R il valore della sottrazione, se c_{out} = 1. R è sovrascritto.
  - In S_1, R viene sempre shiftato a sinistra e concatenato al MSB di A
- Per fondere i due stati servirebbe poter fare le due operazioni in contemporanea
  - Memorizzare la somma nella parte più significativa di R
  - Shiftare il MSB di A nel LSB di R senza che la memorizzazione della differenza lo sovrascriva
- Siccome la differenza riscrive completamente R, è necessario usare un flip-flop rr_0, separato da R per memorizzarne il LSB, proveniente da A
- Nel «nuovo» stato S_1, la gestione di R e rr_0 è:
  - Se c_{out} = 0: shift a sinistra di R e impostare il suo LSB al valore corrente di rr_0
  - Se c_{out} = 1: caricamento in parallelo in R della differenza
  - Shiftare il MSB di A in rr_0 che funge da LSB di R nel confronto con B alla prossima iterazione

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 4 – Divisione binaria «in colonna» V2

- Assumendo $B = 1001$ e $A = 10001100$, fino al ciclo di clock 4 si ha sempre $(R \parallel rr_0) < B \rightarrow (c_{\text{out}} = 0) \rightarrow q_0$ (LSB di Q) e altri shift e concatenazioni
- Dal ciclo di clock 5, $(R \parallel rr_0) > B$ e $c_{\text{out}} = 1$: $(R \parallel rr_0) - B \rightarrow R$ e $(c_{\text{out}} = 1) \rightarrow q_0$
- Il MSB di A viene sempre shiftato in rr_0
- Dopo il ciclo di clock 8, il risultato finale è memorizzato in R e Q (al posto di A)
  - Nota: l'ultimo rr_0 non va considerato come LSB del valore finale di R

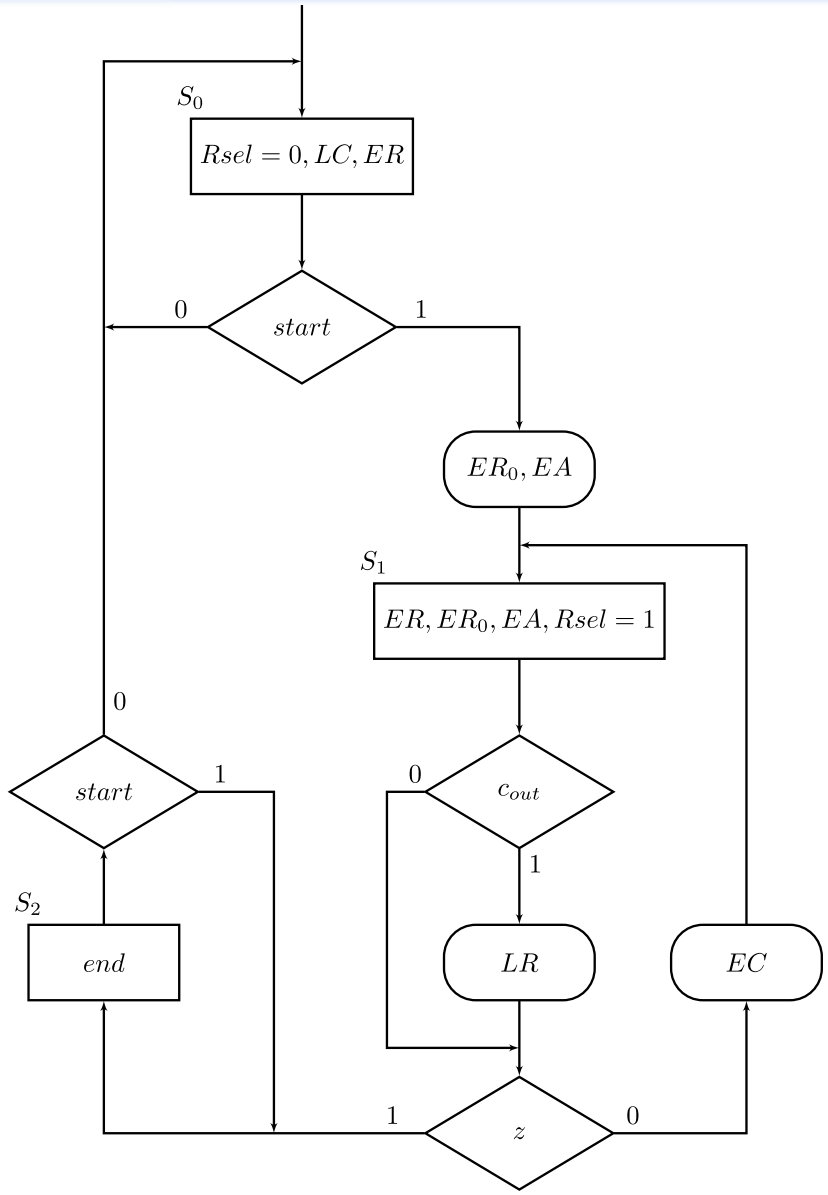
```
    0 0 0 0 1 1 1 1
1 0 0 1 | 1 0 0 0 1 1 0 0
      − 0                    ← 0·1001
        1 0                  10 < 1001
      − 0                    ← 0·1001
        1 0 0                100 < 1001
      − 0                    ← 0·1001
        1 0 0 0              1000 < 1001
      − 0                    ← 0·1001
        1 0 0 0 1            10001 > 1001
      − 1 0 0 1              ← 1·1001
          1 0 0 0 1          10001 > 1001
      − 1 0 0 1              ← 1·1001
            1 0 0 0 0        10000 > 1001
      − 1 0 0 1              ← 1·1001
              1 1 1 0        1110 > 1001
      − 1 0 0 1              ← 1·1001
                1 0 1
```

Clk/Op.		R								rr_0	A/Q							
	Read A, B	0	0	0	0	0	0	0	0	0	1	0	0	0	1	1	0	0
0	Shift L	0	0	0	0	0	0	0	0	1	0	0	0	1	1	0	0	0
1	Shift L, $q_0 \leftarrow 0$	0	0	0	0	0	0	0	1	0	0	0	1	1	0	0	0	0
2	Shift L, $q_0 \leftarrow 0$	0	0	0	0	0	0	1	0	0	0	1	1	0	0	0	0	0
3	Shift L, $q_0 \leftarrow 0$	0	0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0
4	Shift L, $q_0 \leftarrow 0$	0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0	0
5	Subtract, $q_0 \leftarrow 1$	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	1
6	Subtract, $q_0 \leftarrow 1$	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	1
7	Subtract, $q_0 \leftarrow 1$	0	0	0	0	0	1	1	1	0	0	0	0	0	0	1	1	1
8	Subtract, $q_0 \leftarrow 1$	0	0	0	0	0	1	0	1	0	0	0	0	0	1	1	1	1

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 4 – Divisione binaria «in colonna» V2

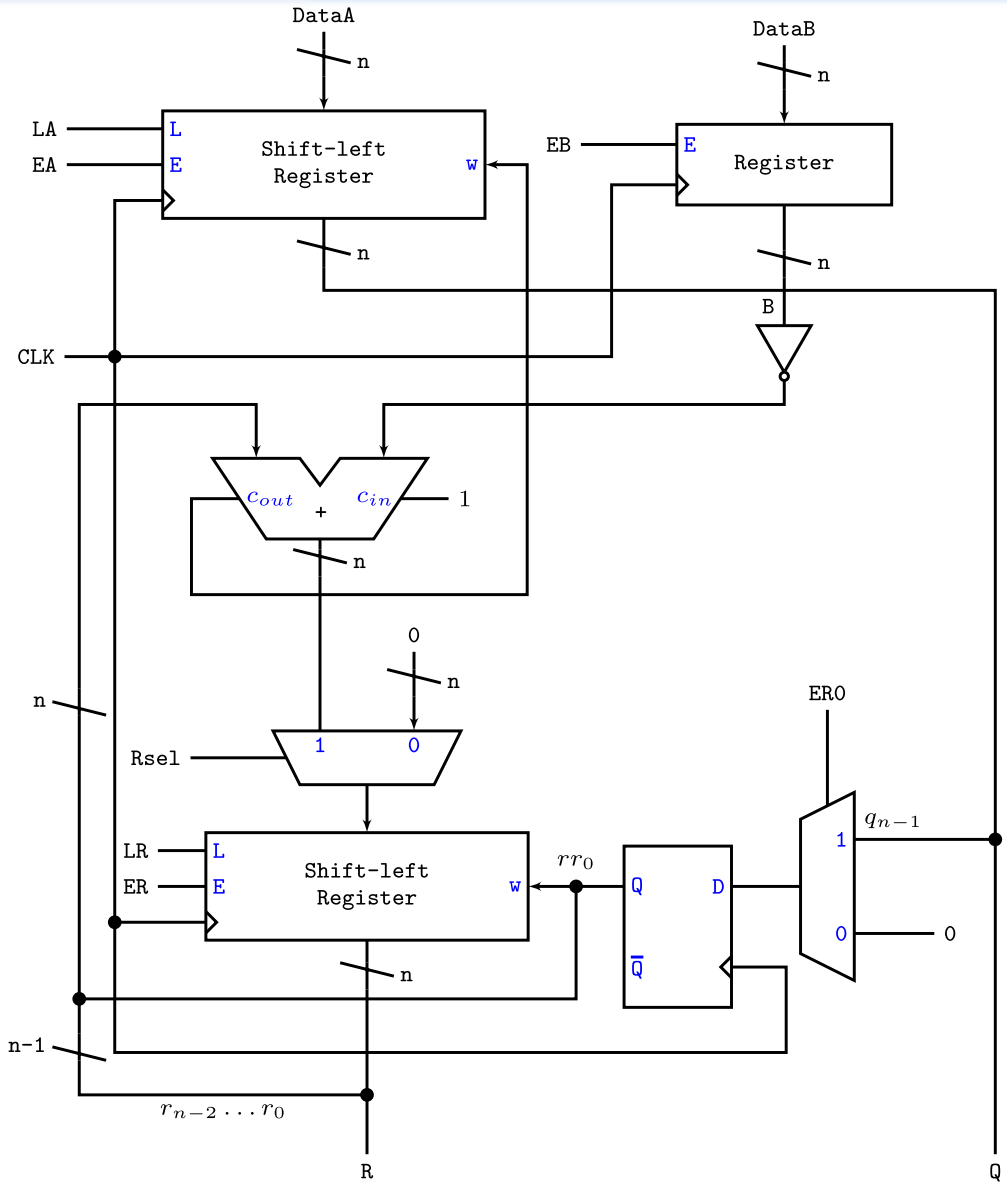
- Nella ASM che rappresenta il solo controllore sono riportati solo i segnali di controllo
- Il segnale ER_0 è usato insieme al flip-flop che memorizza rr_0 per gestire la memorizzazione del MSB di A
- Al ricevimento dello start, si effettua transizione verso S_1 e si ha lo shift a sinistra di R||R0||A
- Nello stato S_1 c_{out} determina il valore con cui verrà caricato R, come descritto in precedenza



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 4 – Divisione binaria «in colonna» V2

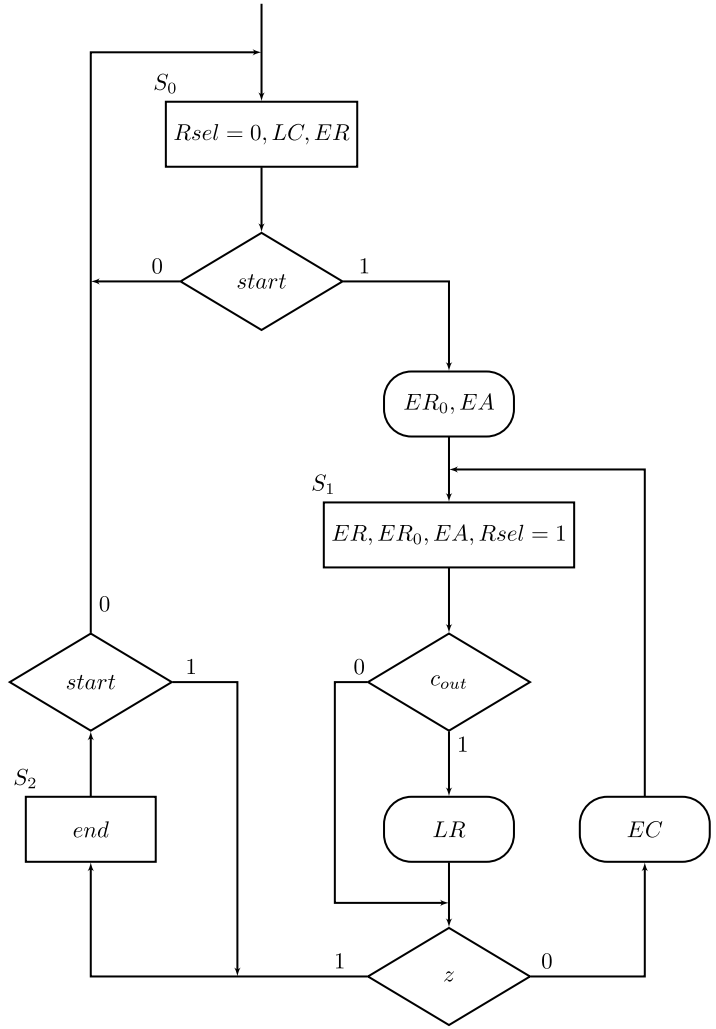
- Il datapath presentato in figura riassume i componenti necessari a modellare il comportamento appena descritto
- Si noti come il valore di Q sia man mano costruito e shiftato nel registro di A
- Si osservi inoltre come uno degli input del sommatore sia costruito prendendo in considerazione la concatenazione degli n-1 bit meno significativi di R con il singolo bit rr_0 come ulteriore LSB

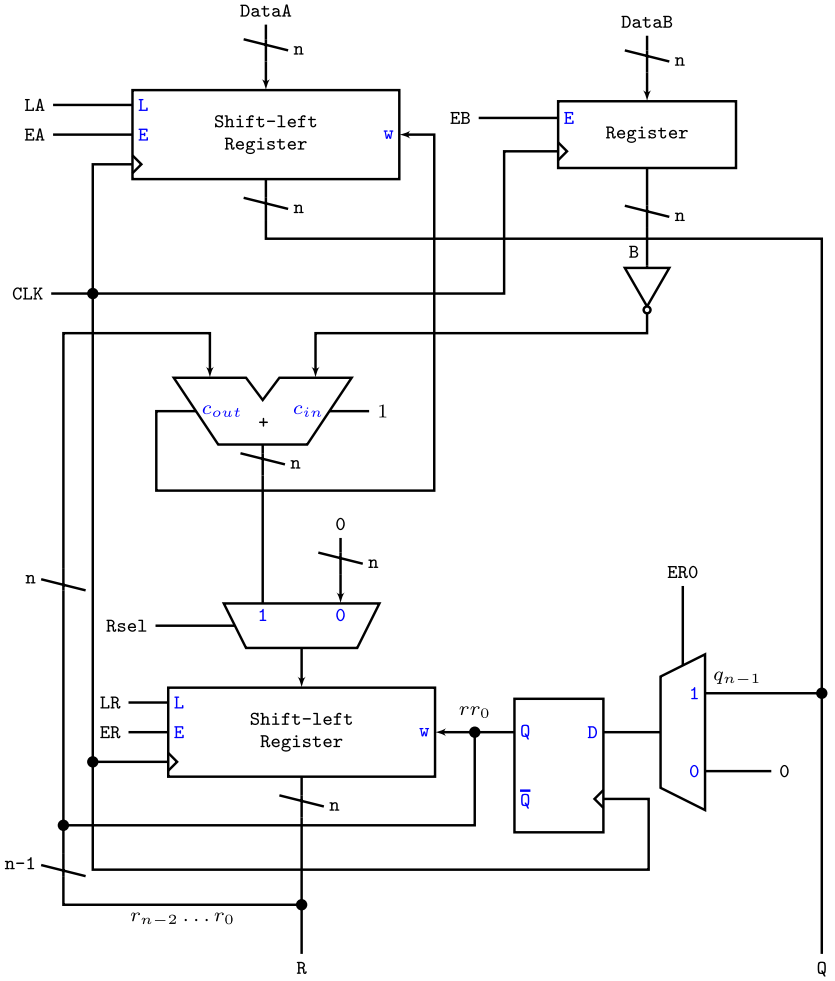


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 4 – Divisione binaria «in colonna» V2

- Affiancando tutti «i pezzi» per avere una migliore visione di insieme…





2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini